

# 2026 비전공 교원 대상 AI 특강 (하계)

## <기계학습 딥러닝 분야>

실습: 2026년 8월 6일 Foundation Model  
소프트웨어학과 김광수 교수  
인공지능학과 장승우 박사과정

Foundation Model

# 환경 설정

---

# Foundation Model

## 실습 환경

### 1 Day2 폴더 열기

| Name | Modified |
|------|----------|
| Day1 | 16h ago  |
| Day2 | 59s ago  |

### 2 00\_install.ipynb 실행

| Name                 | Modified |
|----------------------|----------|
| 00_install.ipynb     | 1m ago   |
| 01_data_preparati... | 4d ago   |
| 02_Image_Inferen...  | 4d ago   |
| 03_Zero_Shot_Cla...  | 2h ago   |
| 04_Fine_Tuning.ip... | 1h ago   |
| 05_LoRA.ipynb        | 1h ago   |
| cat.jpg              | 4d ago   |

### 라이브러리 설치

3

```
!python -m pip install --upgrade pip
!pip install -U "transformers==4.49.0"
!pip install -U opencv-python-headless==4.8.1.78
!pip install timm
!pip install -U bitsandbytes
!pip install --user --ignore-installed -U ipywidgets jupyterlab_widgets
```

4

### 토큰 설정 및 내려받기

```
# ----- 0. Hugging Face 토큰 -----
# 아래 마중표 안에 토큰을 붙여넣으세요 (바탕두면 입력창이 나타납니다)
# 발급: huggingface.co -> 프로필 -> Settings -> Access Tokens -> New token (Read)

HF_TOKEN = "" # 예: "hf_xxxxxxxxxxxxxxxxxxxxxx"

# -----
import os

os.environ["HF_HUB_DISABLE_PROGRESS_BARS"] = "1"

import time, random, getpass
import datasets
from huggingface_hub import snapshot_download, scan_cache_dir
from datasets import load_dataset

token = HF_TOKEN.strip() or os.environ.get("HF_TOKEN", "")

if not token:
    print("토큰을 입력하세요 (없으면 그냥 Enter)")
    print("입력하는 동안 화면에 표시되지 않습니다 - 정상입니다")
    try:
        token = getpass.getpass("HF_TOKEN: ").strip()
    except Exception:
        print("입력창을 사용할 수 없습니다 - HF_TOKEN 변수에 직접 입력하세요")
        token = ""

if token:
    os.environ["HF_TOKEN"] = token
    os.environ["HUGGING_FACE_HUB_TOKEN"] = token # 구버전 호환
    try:
        from huggingface_hub import HfApi
        print("토큰인 유효: ", HfApi().whoami()["name"], "\n")
    except Exception:
        print("토큰 설정됨 (계정 확인 실패 - 토큰을 다시 확인하세요)\n")
else:
    print("토큰이 없거나 잘못되었습니다 - 다시 확인하세요")
```

## Hugging Face 토큰

### 계정 인증 — 요청 제한 완화와 제한 모델 접근

#### 토큰이 필요한 이유

##### ① 요청 제한 완화

- Hub는 5분 단위로 요청 횟수를 제한
- 익명 요청은 IP 기준 집계 — 같은 네트워크끼리 합산
- 다수가 동시에 내려받으면 429 오류 발생
- 인증 시 계정별 할당량 적용

##### ② 제한 모델 접근

- 일부 모델은 사용 조건 동의 필요 (Llama · Gemma 등)
- 모델 페이지에서 동의한 계정의 토큰이 있어야 접근 가능
- 비공개 모델도 권한 있는 계정의 토큰 필요

#### 발급 절차

- ① huggingface.co 가입 · 로그인
- ② 우측 상단 프로필 → Settings
- ③ 좌측 메뉴 Access Tokens
- ④ New token → 이름 입력 → 권한 Read
- ⑤ 생성된 hf\_... 문자열 복사

#### 오류 구분

- 401** — 토큰 미설정 또는 무효
- 403** — 사용 조건 동의 필요
- 429** — 요청 제한, 잠시 후 재시도

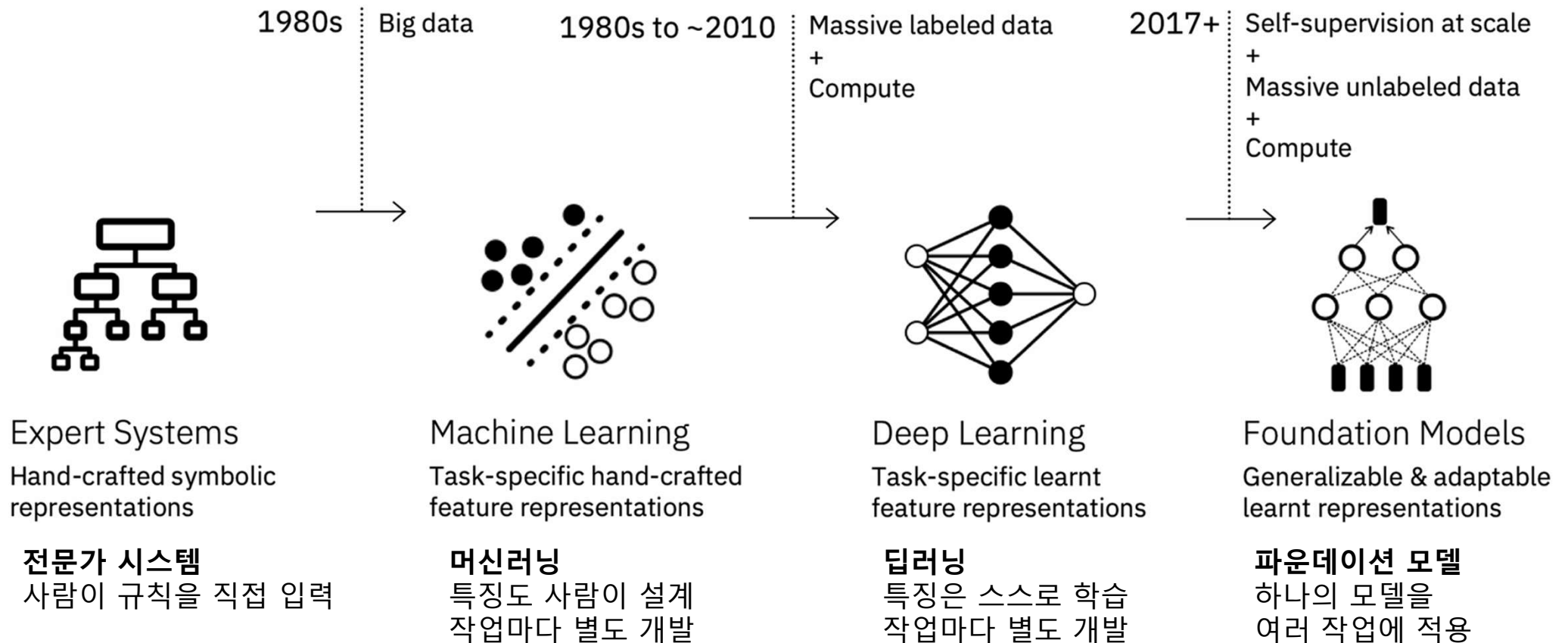
Foundation Model

# 개요

---

## Foundation Model

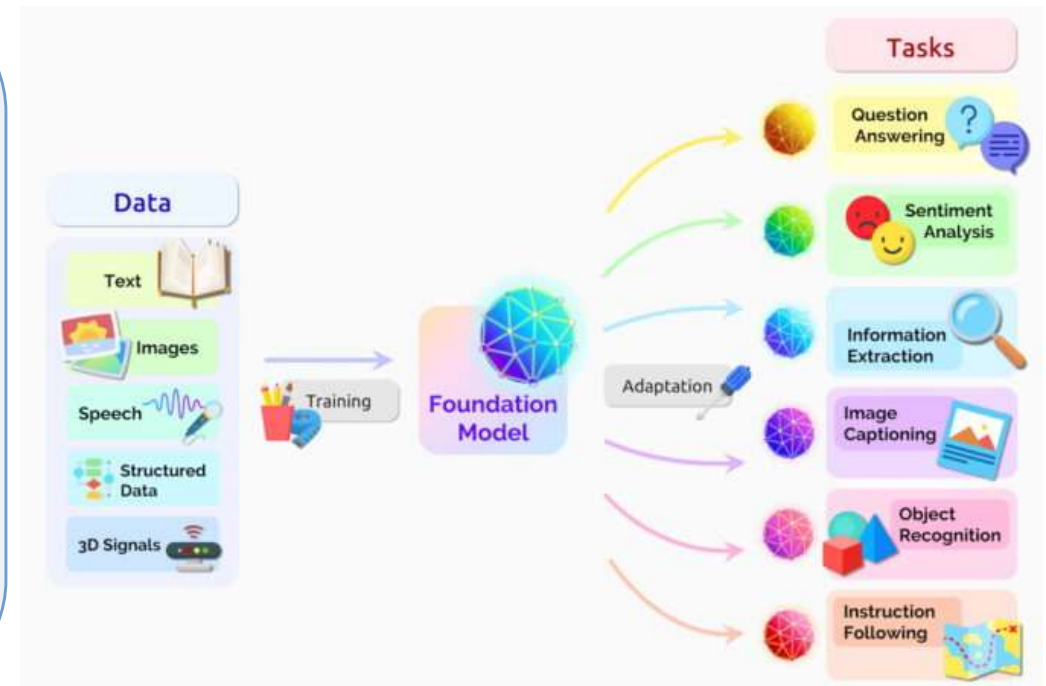
# AI의 발전과 Foundation Model



# Foundation Model이란?

다양한 AI의 기반이 되는 대규모 사전학습 모델

- 광범위한 데이터로 사전학습된 대규모 모델
- 다양한 하위 작업(downstream task)에 적응·활용 가능
- **적응 방식** — 그대로 활용(추론) 또는 미세조정
- 언어·이미지·음성 등 여러 양식(modality)에 적용



# Foundation Model

## 의료영상 분할

SAM은 일반적인 이미지에 강하나 의료영상에는 약함  
→ 처음부터 학습하지 않고, SAM을 의료영상에 맞게 적응시킬 수 있을까?

### 어떻게 확장했나

**토대** — 대규모로 학습된 분할 모델 SAM을 그대로 사용

① **인코더 동결** — 원래 지식 보존, 가벼운 어댑터(LoRA)만 학습

② **방법론 추가** — 2단계 계층적 디코더로 의료 사전지식 반영

③ **프롬프트 불필요** — 전문가 개입 없이 자동 분할

**핵심** — 백본은 그대로, 소수 모듈만 학습 → 적은 데이터로 특화

### 데이터

복부 CT 다장기(Synapse), 심장·전립선 MRI · 8개 장기 · 라벨 극소량(전립선 3 케이스)

### 모델

SAM(ViT 백본) + LoRA 어댑터 + 계층적 디코더 · 인코더 동결

### 결과

슬라이스 10%로 평균 Dice 80.35% (기존 대비 +5%p) · 전체 데이터 시 86.49%, 전용 모델도 능가

| Unlabeled | Labeled | Mean Dice (%) | Methods                      |
|-----------|---------|---------------|------------------------------|
|           |         | 78.27         | SOTA Semi-Supervised Methods |
|           |         | 86.83 ↑       | H-SAM                        |
|           |         | 75.57         | SOTA SAM Variants            |
|           |         | 80.35 ↑       | H-SAM                        |

슬라이스 10%만으로도 기존 SAM 변형들을 능가



## Foundation Model

# 은하 이미지 분류

천문 이미지는 라벨이 적고, 지금껏 대부분 모델을 처음부터 학습 — 자연 이미지로 학습된 공개 파운데이션 모델을 은하 분류에 가져다 쓸 수 있을까?

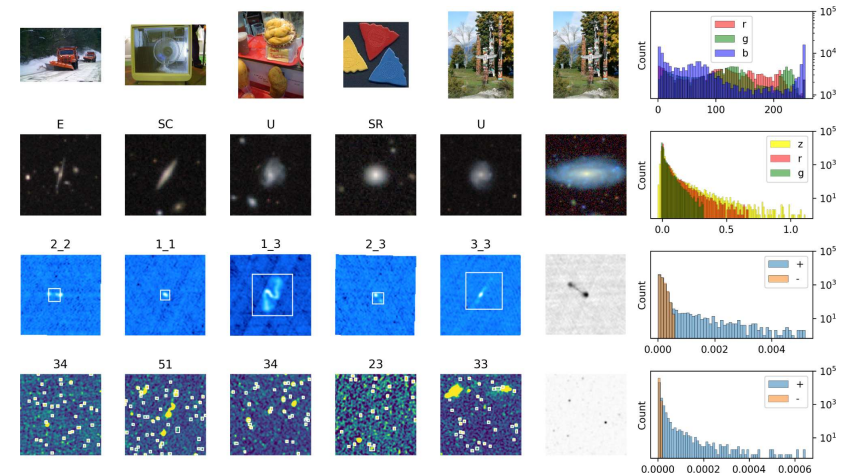
### 어떻게 활용했나

**토대** — 공개된 비전 파운데이션 모델(DINOv2 등)을 그대로 사용

**방식 1** — 백본 동결 + 분류기만 학습 (수십 초면 완료)

**방식 2** — 부족하면 LoRA로 일부만 미세조정 (파라미터 5% 미만)

**핵심** — 처음부터 학습 안 함, 있는 모델을 가져와 적은 라벨로 특화



### 데이터

은하 형태 분류(광학, GalaxyMNIST 1만 장 · 4종) + 전파 은하(RGZ)

### 모델

공개 파운데이션 모델(DINOv2·CLIP 계열, 그대로) + 분류기 / LoRA

### 결과

광학 은하 분류 지도학습 대비 최대 +15%, 라벨 10%로 F1 0.87 · 전파 은하

Examining vision foundation models for classification and detection in optical and radio astronomy, Lastufka et al., Astronomy & Astrophysics 703, A217 (2025)

## Foundation Model

# 동물 행동 분류

카메라 트랩이 만든 수십만 장의 야생동물 사진 — AI 모델을 새로 학습시키지 않고, 공개된 파운데이션 모델을 그대로 가져와 동물의 행동을 분류할 수 있을까?

### 어떻게 활용했나

**토대** — 공개된 비전-언어 파운데이션 모델(CLIP·SigLIP·CogVLM 등 6종)

**방식** — 미세조정 없이(제로샷), 텍스트로 행동을 묻고 이미지와 대조

**예** — "먹는 중 / 이동 중 / 쉬는 중" 중 무엇인지 판별

**추가 실험** — 촬영 시간(주간·야간)·종 이름을 함께 주면 정확도 변화

**핵심** — 학습조차 안 함, 있는 모델에 질문만 던져 활용



### 데이터

알프스 야생동물(CREA Mont-Blanc, 13만여 장) + 세렝게티 · 행동 3종 (먹기·이동·휴식)

### 모델

공개 비전-언어 모델 6종 (CLIP·SigLIP·CogVLM 등), 미세조정 없음

### 결과

전용 학습 없이 최고 모델 96% 정확 · 질문(프롬프트) 방식이 성능에 큰 영향

Zero-shot animal behaviour classification with vision-language foundation models, Dussert et al., Methods in Ecology and Evolution (2025)

## 뇌 시각피질 해석

사람이 사진을 볼 때 뇌는 어떻게 반응할까? — 뇌 예측 모델을 새로 만들지 않고, 공개된 AI 모델(CLIP)을 가져와 뇌 반응을 설명할 수 있을까?

### 어떻게 활용했나

**토대** — 이미지-언어를 함께 학습한 공개 모델 CLIP을 그대로 사용

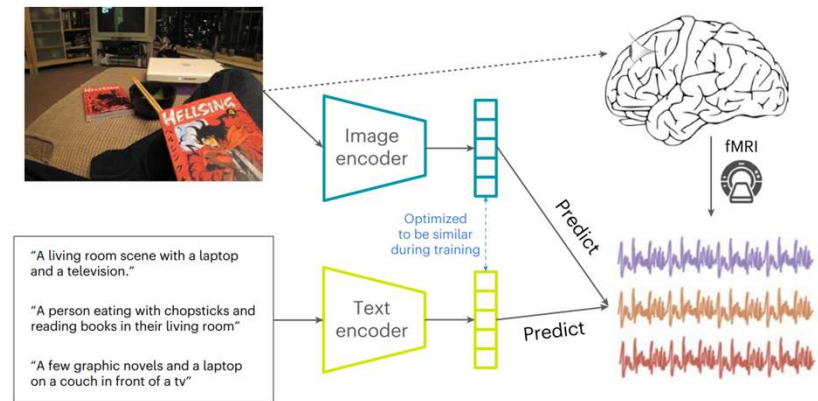
**방식** — 사람에게 보여준 사진을 CLIP에도 입력, 이미지 특징 추출

**학습** — CLIP은 그대로, 뇌 반응 예측 모델만 학습

**비교** — 언어 없이 이미지만 학습한 기존 모델과 대조

**발견** — 언어까지 배운 AI가 인간 뇌 시각 처리를 더 잘 닮음

**핵심** — AI를 만든 게 아니라, 뇌를 이해하는 도구로 활용



CLIP 기반 모델이 고차 시각피질 반응의 최대 79%를 설명

### 데이터

사람들이 실제 사진을 볼 때 측정한 뇌 반응(fMRI, Natural Scenes Dataset)

### 모델

공개 CLIP(그대로) + 뇌 반응 예측 모델(복셀 단위)

### 결과

고차 시각피질 반응 최대 79%(R<sup>2</sup>) 설명 · 언어 없이 학습한 기존 모델을 크게 능가

Better models of human high-level visual cortex emerge from natural language supervision with a large and diverse dataset, Wang et al., Nature Machine Intelligence (2023)

## Foundation Model Hugging Face란?



# Hugging Face

전 세계 AI 모델을 모아 제공하는 개방형 플랫폼

- 기업·대학·연구기관이 개발한 모델을 무료 공개
- 범용 모델과 분야 특화 모델(의료·위성·생명 등)을 모두 제공

**전문 지식 없이 사용**  
만들어진 모델을 그대로 활용

**최신·다양한 모델**  
내 분야 모델을 찾아 바로 사용

**직접 내려받아 실행**  
데이터 외부 유출 없음

**무료 · 오픈 라이선스**  
연구 · 교육 목적 대부분 무상 이용

## Hub에는 무엇이 있나

Hugging Face Hub — 모델·데이터·데모가 모인 개방형 플랫폼

### Models

#### 미리 만들어진 AI 모델

- 이미지·텍스트·음성 등 작업별 최신 모델
- 모델 카드로 용도·한계·편향 안내
- 작업·언어별로 검색 가능

### Datasets

#### 학습·평가용 공개 데이터

- 다양한 분야·형식의 데이터
- 데이터셋 카드로 출처·구성 설명
- 브라우저에서 데이터 미리 보기

### Spaces

#### 브라우저 체험 데모

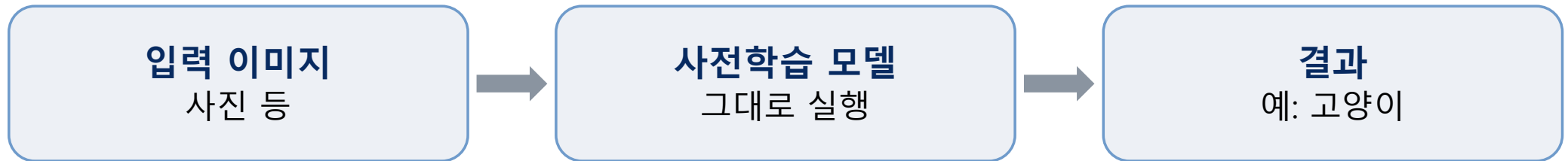
- 설치 없이 바로 실행해보는 앱
- 모델을 직접 눌러보며 확인
- 코드 없이 결과 체험

규모 — 모델 240만 · 데이터셋 73만 · 데모 100만 개 이상 (2026년 기준, 모두 공개)

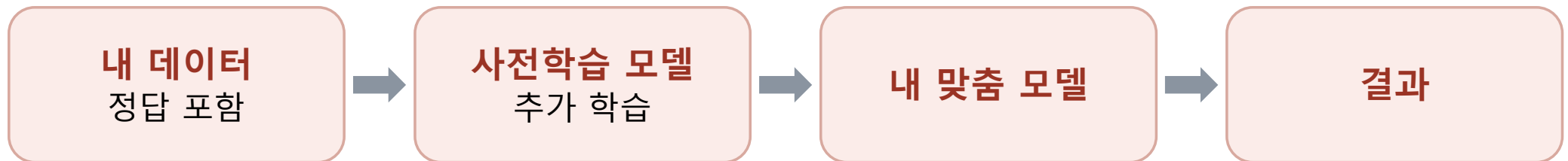
## 추론과 미세조정

모델을 쓰는 방법은 두 가지 — 추론과 미세조정

### 추론



### 미세조정



## 언제 무엇을 쓰나

### 상황별 선택 기준

#### 추론이면 충분한 경우

- 일반적인 작업 (흔한 사물·동물 등)
- 잘 맞는 모델이 이미 존재
- 빠른 결과만 필요

#### 미세조정이 필요한 경우

- 특수한 분야 (특정 세포·부품·문서 등)
- 기존 모델이 내 데이터에 부정확
- 내 기준·라벨로 분류 필요

#### 판단 순서

- 먼저 추론 시도 → 부족하면 미세조정
- 직접 학습은 맞는 모델이 없을 때만

Foundation Model

# 데이터 업로드

---



## 실습 데이터 준비

### 세 가지 데이터 준비 방법

#### URL 입력

웹 이미지 주소를 코드에  
직접 지정

▶ 준비 없이 즉시 시작

#### Hub 데이터셋

공개 데이터셋을 내려받아  
사용

▶ 라벨 포함 데이터 필요

#### 직접 업로드

보유 이미지를 작업 폴더에  
올려 사용

▶ 연구 데이터 활용

## Foundation Model

# URL 입력

### 웹 이미지 주소를 파일 경로 대신 직접 지정

#### 주소 확보

- 웹 이미지에서 우클릭 → 이미지 주소 복사
- 주소가 .jpg · .png 로 끝나는지 확인
- 페이지 주소가 아닌 이미지 자체의 주소 사용
- 로그인에 필요한 사이트는 접근 불가

#### 불러오기

```
import requests
from PIL import Image

url = "https://.../cat.jpg"
image = Image.open(requests.get(url, stream=True).raw)
```

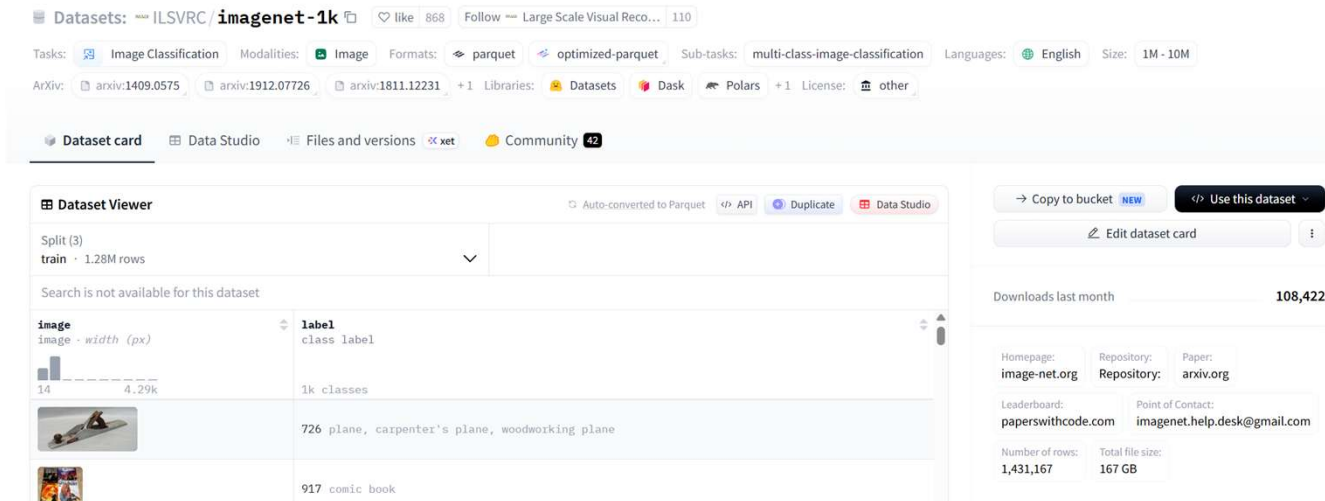
- 주소에서 이미지를 받아 그대로 열기
- 업로드 과정 불필요
- 화면에 이미지가 표시되면 정상



# Foundation Model

## Hub 데이터셋 찾기

### Datasets 탭에서 공개 데이터셋 검색·확인



### 검색 절차

- 1 huggingface.co → Datasets 탭
- 2 좌측 Tasks 필터에서 작업 선택 (Image Classification 등)
- 3 정렬 기준 선택 — 다운로드 · 트렌딩 · 최신
- 4 데이터셋 카드에서 용도·출처·라이선스 확인

### 선택 시 확인 사항

- **미리보기(Data Studio)** — 브라우저에서 내용 직접 확인
- **규모** — 이미지 수, 파일 용량
- **구성** — 이미지와 라벨의 열 이름
- **이름 형식** — 제작자/데이터셋명

## 데이터셋 불러오기

### load\_dataset 함수로 Hub 데이터셋 내려받기

```
from datasets import load_dataset

ds = load_dataset(
    "AI-Lab-Makerere/beans",
    split="train"
)
```

**첫 인자** — 데이터셋 이름 (제작자/데이터셋명)

**split** — 사용할 구간 지정

### split 의미

- **train** — 학습용 구간
- **validation** — 검증용 구간
- **test** — 시험용 구간
- 생략 시 전체 구간을 함께 불러옴
- 구간 구성은 데이터셋마다 상이 — 카드에서 확인

- 첫 실행은 내려받기로 시간 소요 — 이후 저장된 파일 재사용
- 예시 데이터셋 — 콩잎 병해 이미지 3종 분류

## 데이터셋 구조 확인

불러온 데이터의 개수와 구성 항목 파악

```
ds

# 출력 예시
Dataset({
  features: ['image', 'labels'],
  num_rows: 1034
})
```

### 출력 항목

- **features** — 각 항목의 구성 열
  - **image** — 이미지 데이터
  - **labels** — 정답 분류 번호
- **num\_rows** — 전체 이미지 수
- 열 이름은 데이터셋마다 상이 — label · labels 등
- 이후 코드에서 이 열 이름을 그대로 사용

- 구조 확인은 필수 절차 — 열 이름을 알아야 이미지·라벨 접근 가능
- 개별 항목 확인 — **ds[0]**

## 이미지와 라벨 확인

개별 항목에서 이미지와 정답 추출

### 이미지 확인

```
image = ds[0]["image"]  
image
```

- `ds[0]` — 첫 번째 항목 선택
- `["image"]` — 이미지 열 지정
- 마지막 줄에 변수명만 입력 → 화면에 표시

### 라벨 확인

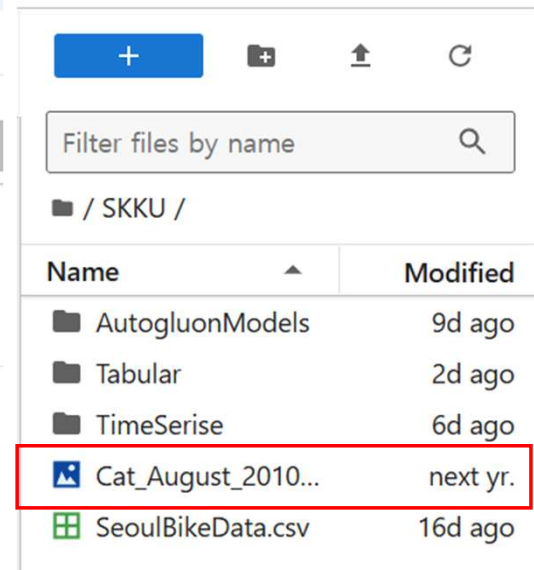
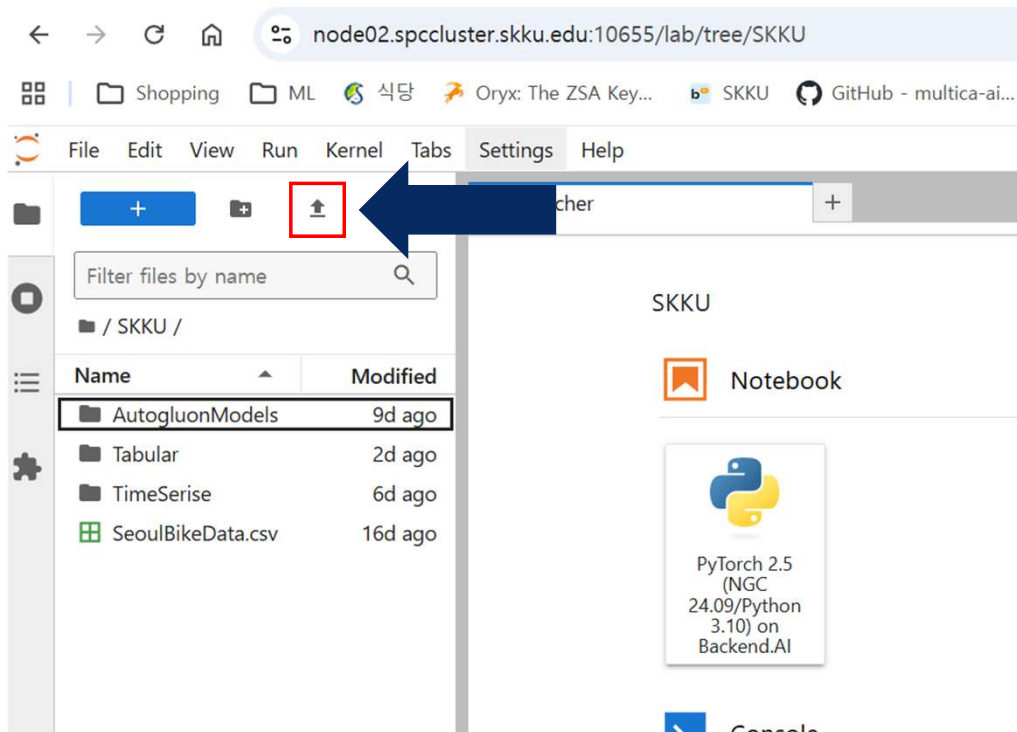
```
ds[0]["labels"]  
ds.features["labels"].names
```

- 첫 번째 줄 — 정답 분류 번호 (예: 0)
- 두 번째 줄 — 번호에 대응하는 이름 목록
- 번호와 이름을 함께 확인해야 결과 해석 가능

## Foundation Model

# 직접 업로드

이미지를 작업 폴더에 업로드

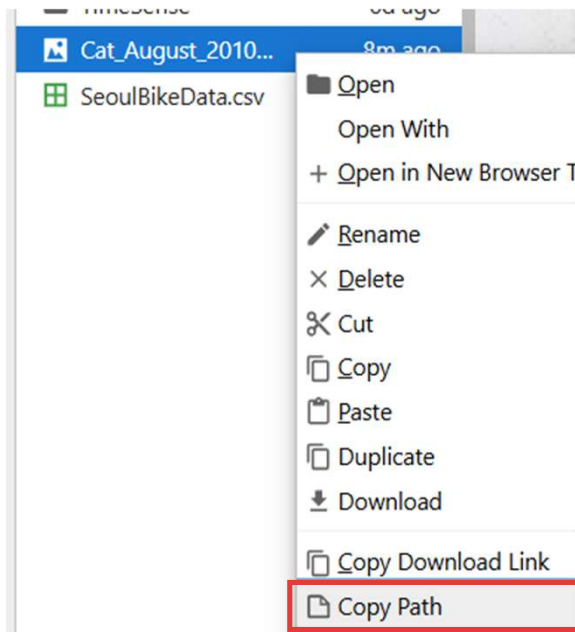


### 절차

- ① 좌측 파일 탐색기에서 작업 폴더 확인
- ② 업로드 버튼 클릭 또는 파일을 끌어다 놓기
- ③ 목록에 파일이 나타나면 업로드 완료
- ④ 파일명 확인 — 코드에서 그대로 사용

# 업로드한 이미지 열기

파일 경로를 복사하여 이미지 확인



### 경로 복사

- ① 파일 우클릭 → 메뉴 하단 Copy Path 선택
- ② 복사된 경로를 따옴표 안에 붙여넣기
- ③ 경로 예시 — "SKKU/Cat\_August\_2010.jpg"

### 불러오기

```
from PIL import Image

image = Image.open("Cat_August_2010.jpg")
image
```

- **Image.open** — 파일을 이미지로 불러오기
- 마지막 줄에 변수명만 입력 → 화면에 이미지 표시
- 이미지가 보이면 경로 정상



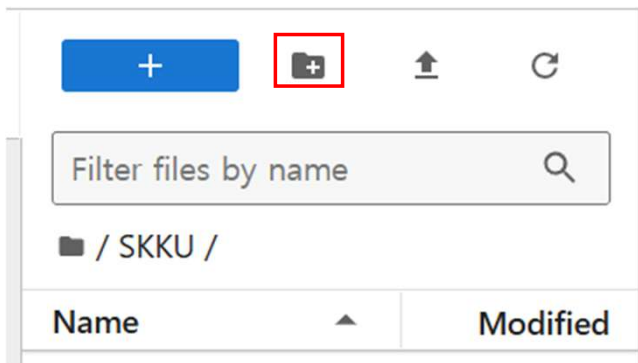
## Foundation Model

# 폴더 단위 업로드 방법

폴더 생성 후 업로드 또는 ZIP 압축 후 해제

### 방법 1 폴더 생성 후 업로드

- ① 파일 탐색기에서 **새 폴더** 버튼 클릭
- ② 폴더 이름 입력 — **영문 권장** (예: dataset)
- ③ 폴더 더블클릭 후 진입
- ④ **업로드 버튼(↑)**으로 이미지 선택 — 여러 개  
동시 선택 가능



### 방법 2 압축 파일 업로드

- ① 내 PC에서 이미지 폴더를 **ZIP**으로 압축 —  
우클릭 → 압축
- ② ZIP 파일 **단일 업로드**
- ③ 노트북에서 **압축 해제**

### 실행 압축 해제 코드

```
import zipfile

with zipfile.ZipFile("dataset.zip") as z:
    z.extractall("dataset")
```

## Foundation Model

# 폴더 구조가 데이터셋 구조를 결정

폴더를 나누면 폴더 이름이 정답 라벨이 된다

### ① 한 폴더에 모은 경우

```
dataset/  
  a.jpg    b.jpg  
  c.jpg  
features: ['image']
```

- 구분 근거 없음 — 라벨 열 미생성
- 이미지만 불러와짐
- 정답이 불필요한 추론에 사용

### ② 라벨별로 나눈 경우

```
dataset/  
  cat/    a.jpg    b.jpg  
  dog/    c.jpg  
features: ['image', 'label']
```

- 폴더 이름이 정답 라벨로 인식 — cat · dog
- 사진마다 소속 폴더 이름이 정답
- 학습(미세조정)에 사용

```
# 두 경우 모두 동일 — 폴더 경로만 지정  
from datasets import load_dataset  
ds = load_dataset("path/to/folder")  
ds = load_dataset("imagefolder", data_dir="path/to/folder")
```

# 학습용·평가용 데이터의 사전 분리

## 학습용과 평가용 데이터의 사전 분리

### 폴더 구조

```
dataset/  
  train/  
    cat/  
    dog/  
  test/  
    cat/  
    dog/
```

- 상위 폴더 — **구간** (train · test)
- 하위 폴더 — **라벨** (cat · dog)
- 두 계층 모두 필요

```
ds["train"]  
ds["test"]
```

- 구간별로 나누어 접근

### 구간을 나누는 이유

- **train** — 모델 **학습**에 사용하는 데이터
- **test** — 학습에 쓰지 않고 **성능 평가**에만 사용
- 학습 데이터로 평가 시 **성능 과대평가** — 학습한 내용의 반복에 불과
- 미사용 데이터로 평가해야 **일반화 성능** 확인 가능
- 검증용(**validation**) 구간을 추가로 두기도 함

## Foundation Model

# metadata 파일 — 폴더 구조로 표현되지 않는 정보 지정

file\_name 열로 이미지와 정보를 연결

### 배치와 작성

```
dataset/  
  metadata.csv  
  0001.png
```

```
# metadata.csv  
file_name,species,date  
0001.png,cheetah,2024-05-01  
0002.png,zebra,2024-05-03
```

- 이미지와 같은 폴더에 배치
- 지원 형식 — .csv · .jsonl · .parquet

### 작성 규칙

- file\_name 열 필수 — 이미지와 정보를 연결
- 값은 metadata 파일 기준 상대 경로
- \*\_file\_name 형태도 사용 가능
- 나머지 열 이름이 그대로 데이터셋 열이 됨

```
features:  
  ['image', 'species', 'date']  
ds[0]["species"] # 'cheetah'
```

## Foundation Model

# 작업 유형에 따른 정답 기재 방식

분류는 폴더 이름, 그 외 작업은 metadata 파일

### 이미지 캡셔닝

```
# metadata.csv
file_name,text
0001.png,A golden retriever...
0002.png,A german shepherd
```

```
features: ['image', 'text']
```

- 이미지를 설명하는 **문장** 기재
- **text** 열 이름 → 데이터셋의 **text** 열
- 확인 — **ds[0]["text"]**

### 객체 탐지

```
# metadata.jsonl
{"file_name": "0001.png",
 "objects": {"bbox":
  [[302,109,73,52]],
 "categories": [0]}}
```

```
features: ['image', 'objects']
```

- **bbox** — 대상의 위치 좌표
- **categories** — 대상의 범주 번호
- 대상이 여러이면 목록으로 나열
- 값 구조가 복잡하여 **JSONL** 형식 사용

## Foundation Model

# 업로드한 데이터로 실행

불러온 데이터셋을 추론과 학습에 그대로 사용

### 추론에 사용

```
from datasets import load_dataset
from transformers import pipeline

ds = load_dataset("dataset")
clf = pipeline("image-classification")
```

```
clf(ds["train"][0]["image"])
```

- 폴더 경로만 지정 — Hub 데이터셋과 동일한 방식
- 추론은 라벨 없이 이미지만으로 가능
- 결과 확인 — ds[0]["image"]

### 학습(미세조정)에 사용

```
labels = ds["train"].features["label"].names

trainer = Trainer(
    model=model,
    train_dataset=ds["train"],
    eval_dataset=ds["test"]
)
```

```
trainer.train()
```

- **추론** — 이미지만 있으면 됨, 라벨 폴더 불필요
- **학습** — train · test 구간과 정답 라벨 필요
- Hub 데이터셋과 사용법 동일 — 이름 자리에 폴더 경로
- 이후 절차는 앞 실습과 같음

Foundation Model

추론

---

# 추론

학습이 완료된 모델의 파라미터를 고정한 채, 새로운 입력에 대한 예측을 산출하는 과정

입력 이미지

사전학습 모델

예측 결과

### 추론의 정의

- **파라미터 고정** — 모델 내부 값(가중치)을 갱신하지 않음
- **순전파만 수행** — 입력에서 출력 방향으로 1회 계산
- **학습 과정 부재** — 정답 라벨·오차 역전파 불필요
- 필요 입력은 모델과 입력 이미지뿐

### 사전학습 모델의 활용

- **파운데이션 모델** — 대규모 데이터로 사전학습 (pre-training) 완료
- 형태·질감·객체에 대한 범용 표현 (representation) 보유
- 추가 학습 없이 미보유 데이터에 일반화 가능
- 출력은 각 후보에 대한 예측 확률로 산출



## Hugging Face의 추론

pipeline — 사전학습 모델을 추론에 사용하는 표준 도구

### pipeline의 정의

- 복잡한 과정을 감추고 작업별 단순 사용법 제공
- **지정 항목은 세 가지** — 작업(task) · 모델 · 입력
- 작업만 지정하면 기본 모델 자동 준비
- 원하는 모델을 직접 지정할 수도 있음

### 추론 처리 과정

- **전처리** — 이미지를 모델 입력 형식으로 변환
  - **모델 실행** — 사전학습된 지식으로 예측 산출
  - **후처리** — 예측값을 사람이 읽는 형태로 정리
- 사용자는 입력만 제공, 전 과정 자동 수행

- 작업 이름 변경만으로 분류·탐지·세그멘테이션 수행 — 사용법은 동일
- 모델 파라미터는 변경되지 않는 추론 전용 도구
- 미세조정은 별도 도구(Trainer)로 수행

## Foundation Model

# pipeline이 지원하는 작업

작업(task) 이름 변경만으로 다양한 작업 수행

### 비전 (Vision)

**image-classification** — 이미지 분류  
**object-detection** — 객체 탐지  
**image-segmentation** — 세그멘테이션  
**depth-estimation** — 깊이 추정  
**image-to-text** — 이미지 설명 생성  
**mask-generation** — 마스크 생성  
**video-classification** — 영상 분류

### 멀티모달 (Multimodal)

**zero-shot-image-classification** — 제로샷 이미지 분류  
**zero-shot-object-detection** — 제로샷 객체 탐지  
**visual-question-answering** — 이미지 질의응답  
**document-question-answering** — 문서 질의응답  
**zero-shot-audio-classification** — 제로샷 음성 분류

그 외 텍스트(분류·요약·번역·생성·질의응답) · 음성(인식·합성) 분야 15개

## Foundation Model

# 실행 예시 — 이미지 분류

### 세 줄로 이미지의 종류를 판별

```
from transformers import pipeline
clf = pipeline("image-classification")
clf("cat.jpg")
```

1행 — pipeline 불러오기

2행 — 작업 이름 지정 (기본 모델 자동 준비)

3행 — 이미지 입력



### 결과 읽기

- 출력 형식 — 라벨(label)과 점수(score)의 목록
- 예: `tabby 0.51 / tiger cat 0.24`
- 점수 — 각 후보에 대한 예측 확률 (1에 가까울수록 확신 ↑)
- 상위 후보가 함께 제시됨 — 단일 정답이 아님

Foundation Model

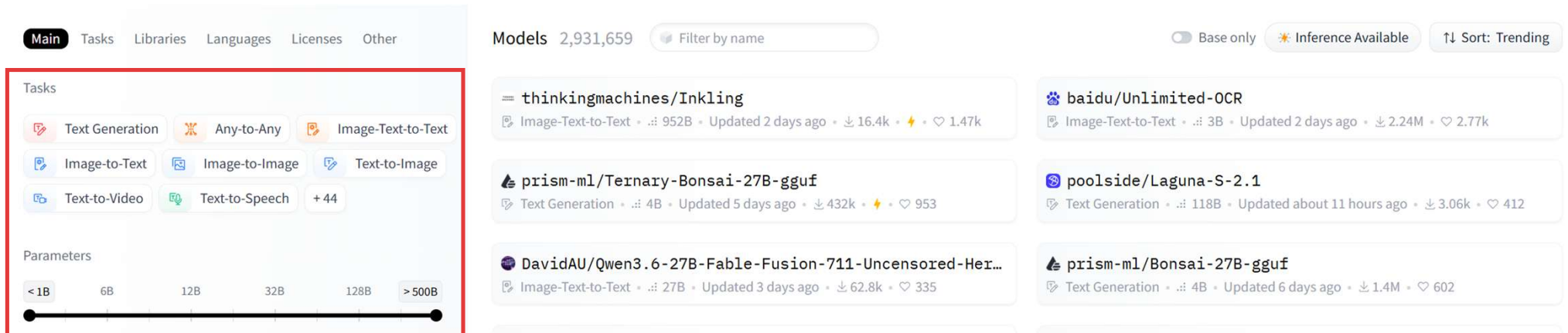
# 추론 실습 1

---

# Foundation Model

## Hub에서 모델 찾기

필터와 정렬로 목적에 맞는 모델 검색



### 검색 절차

- ① huggingface.co → Models 탭
- ② 작업(Tasks) 필터에서 수행할 작업 선택
- ③ 파라미터 수 필터로 모델 규모 조절
- ④ 정렬 기준 선택 — 다운로드 · 트렌딩 · 최신
- ⑤ 이름 형식 확인 — 제작자/모델명

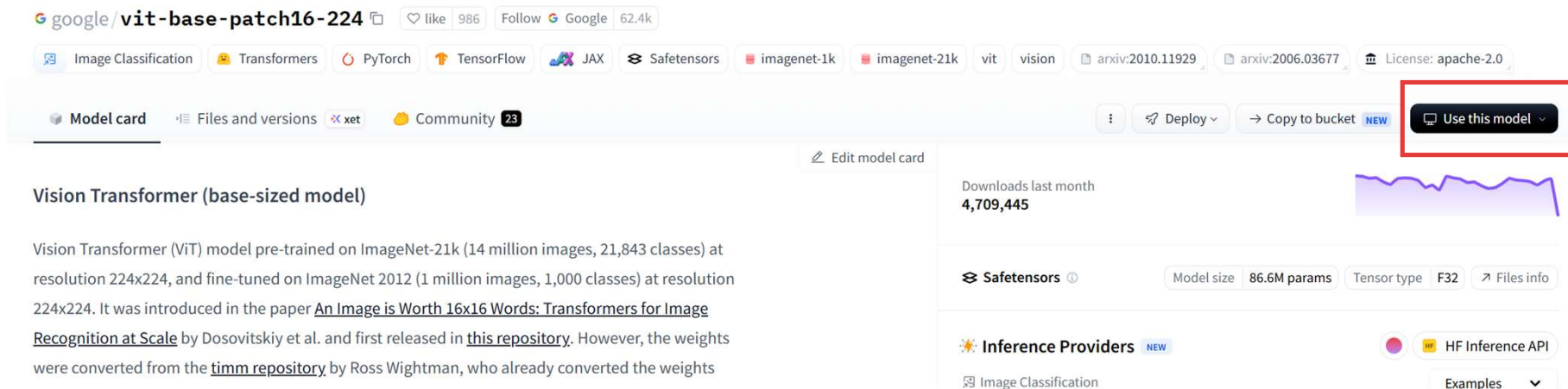
### 모델 카드 확인

- 용도 — 의도된 사용 목적
- 학습 데이터 — 어떤 데이터로 학습되었는가
- 한계와 편향 — 무엇을 못하는가, 어떤 치우침이 있는가
- 평가 결과 — 보고된 성능 수치
- 라이선스 — 사용 조건

## Foundation Model

# 모델을 코드에 지정

Use this model 버튼으로 코드 복사



### 복사 절차

- ① 모델 페이지 우측 상단 **Use this model** 클릭
- ② 목록에서 **Transformers** 선택
- ③ 표시된 코드에서 모델 이름 확인
- ④ 코드의 `model=` 자리에 붙여넣기

### 이미지 분류 모델 예시

- **google/vit-base-patch16-224** — 표준 분류 모델
- **microsoft/resnet-50** — 널리 사용되는 기본 모델
- **facebook/deit-base-patch16-224**
- 세 모델 모두 ImageNet 1000종 분류 학습

## Foundation Model

# 이미지 분류란

이미지 전체에 정해진 분류 중 하나의 이름을 붙이는 작업



### 정의

- 사진 한 장 전체를 보고 하나의 범주로 판단
- **분류 범위** — 모델이 학습한 이름 목록 안에서만 선택
- **세그멘테이션과 차이** — 부분이 아닌 전체를 판단

### 결과 형식

- 이름(label)과 점수(score)의 목록
- 점수가 높은 순으로 상위 후보 함께 제시

### 활용 분야

- **의료** — 영상에 라벨을 붙여 질병 징후 확인
- **환경** — 위성영상으로 삼림 감시 · 산불 탐지
- **농업** — 작물 상태 · 토지 이용 판별
- **생태** — 동식물 종 식별

## Foundation Model

# 실습 — 이미지 분류

작업과 모델을 지정하여 이미지의 종류를 판별

```
# python
from transformers import pipeline
from PIL import Image

image = Image.open("cat.jpg")

clf = pipeline(
    task="image-classification",
    model="google/vit-base-patch16-224",
    device=0
)
```

**task** — 수행할 작업 이름  
**model** — 앞서 선택한 모델  
**device** — 연산 장치 (0=GPU, -1=CPU)  
**clf(image)** — 준비한 이미지 입력

### 작업 (task)

필수 · 수행 내용 결정

### 모델 (model)

선택 · 생략 시 기본 모델 자동 준비

### 입력

경로 · 주소 · 이미지 객체 모두 가능

### 장치 (device)

선택 · 미지정 시 CPU에서 실행



# 결과 읽기

### 출력은 라벨과 점수의 목록

```
[{'score': 0.8821496367454529, 'label': 'Egyptian cat'},  
{ 'score': 0.07465527951717377, 'label': 'tabby, tabby cat'},  
{ 'score': 0.037991415709257126, 'label': 'tiger cat'},  
{ 'score': 0.0009538870654068887, 'label': 'lynx, catamount'},  
{ 'score': 0.00010449632100062445, 'label': 'tiger, Panthera tigris'}]
```

**label** — 모델이 예측한 분류 이름

**score** — 해당 분류에 대한 예측 확률

점수가 높은 순으로 정렬

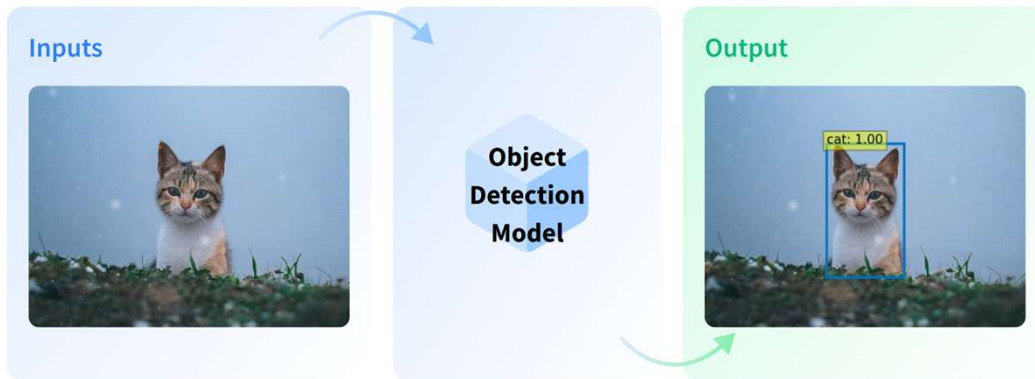
기본 5개 후보 출력

### 결과 해석

- **점수** — 1에 가까울수록 확신이 큼 (전체 합 1)
- **상위 후보가 함께 제시** — 단일 정답이 아님
- **라벨은 영어로 출력** — 모델의 학습 분류 체계를 따름
- 상위 후보가 서로 유사하면 모델이 판단을 나눈 것

# 객체 탐지란

이미지 속 대상의 종류와 위치를 함께 찾는 작업



### 정의

- 대상마다 종류와 위치를 하나씩 출력
- **분류와 차이** — 전체가 아닌 개별 대상 단위
- 같은 종류가 여러 개면 각각 따로 검출

### 결과 형식

- 이름 · 점수 · 위치 상자(box)
- 상자는 좌표 네 개 — 좌측 상단이 기준점

### 활용 분야

- **자율주행** — 보행자 · 표지판 · 신호등 인식
- **의료 영상** — 병변 위치 표시
- **감시 · 보안** — 출입 인원 · 이상 상황 확인
- **검색** — 특정 대상이 있는 이미지 찾기

# 객체 탐지

작업 이름 변경만으로 대상의 위치까지 탐지

### 코드

```
from transformers import pipeline
from PIL import Image
image = Image.open("cat.jpg")
det = pipeline(
    task="object-detection",
    model="facebook/detr-resnet-50"
)
result = det(image)
result
```

- 앞 실습과 동일한 구조 — 작업 이름과 모델만 변경
- 이미지는 앞서 준비한 변수 그대로 사용

### 출력 형태

```
[{'score': 0.9987272620201111,
  'label': 'cat',
  'box': {'xmin': 176, 'ymin': 456,
          'xmax': 3496, 'ymax': 1946}}]
```

- **label** · **score** — 분류와 동일
- **box** — 대상의 위치 (사각형 좌표)
- 이미지 속 대상마다 하나씩 출력

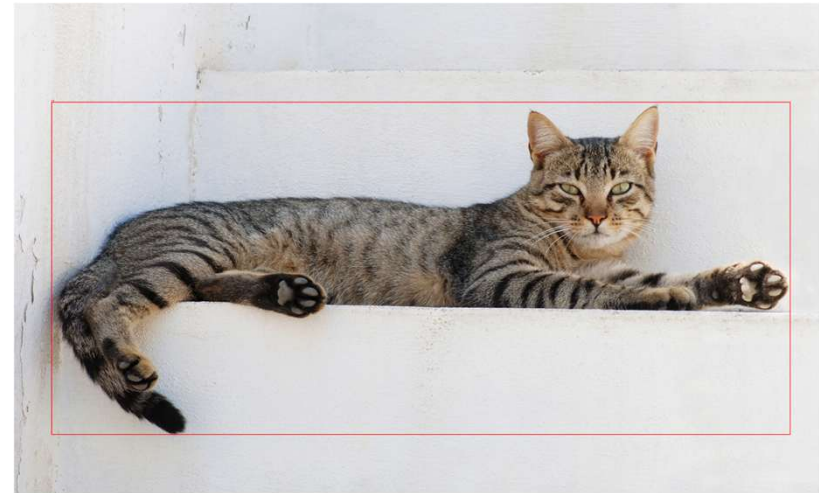
## Foundation Model

# 탐지 결과 시각화

좌표를 사각형으로 그려 위치 확인

```
from PIL import ImageDraw

draw_image = image.copy()
draw = ImageDraw.Draw(draw_image)
for r in result:
    box = r["box"]
    draw.rectangle(
        (box["xmin"], box["ymin"],
         box["xmax"], box["ymax"]),
        outline="red", width=3
    )
    draw.text(
        (box["xmin"], box["ymin"] - 12),
        r["label"], fill="red"
    )
```

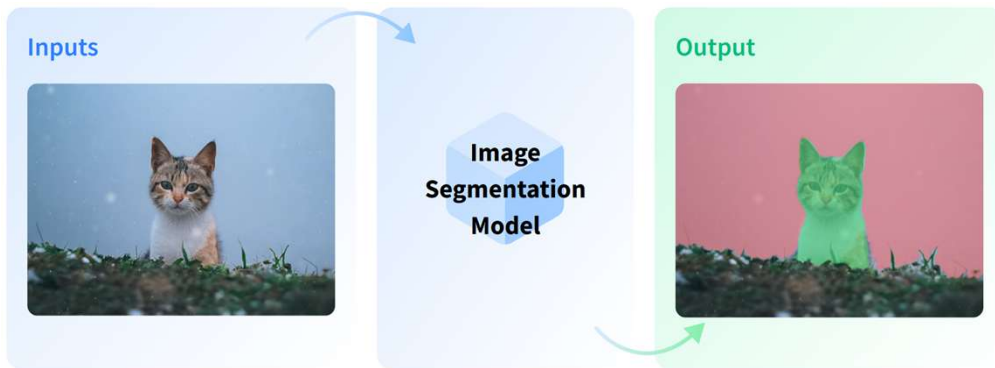


- **image.copy()** — 원본 보존을 위한 사본 생성
- **ImageDraw.Draw** — 이미지에 그림을 그리는 도구
- **for r in result** — 탐지된 대상마다 반복
- **draw.rectangle** — 좌표 네 개로 사각형 그리기
- **draw.text** — 상자 위에 분류 이름 표시
- **outline · width** — 선 색상과 두께

## Foundation Model

# 세그멘테이션이란

이미지를 영역으로 나누어 모든 픽셀을 대상에 대응시키는 작업



### 정의

- 이미지의 모든 픽셀을 각각 어떤 대상에 속하는지 대응
- **탐지와 차이** — 사각형이 아닌 대상의 실제 윤곽
- 결과는 영역을 칠한 마스크 이미지로 출력

### 활용 분야

- **의료 영상** — 장기 · 조직 구분, X선 · MRI 분석
- **자율주행** — 차선 · 장애물 등 도로 구조 인식
- **배경 제거** — 대상만 남기고 나머지 분리

### 연구 활용

- **면적 산출** — 병변 · 균열 · 경작지 등 넓이 계산
- **경계 추출** — 불규칙한 형태의 윤곽 확보

## Foundation Model

# 세그멘테이션

사각형이 아닌 대상의 실제 윤곽을 따라 픽셀 단위로 구분

### 코드

```
from transformers import pipeline
from PIL import Image

image = Image.open("cat.jpg")

seg = pipeline(
    task="image-segmentation",
    model="facebook/detr-resnet-50-panoptic"
)

result = seg(image)
result
```

- 앞 실습과 동일한 구조 — 작업 이름과 모델만 변경

### 출력 형태

```
[{'score': 0.998772,
  'label': 'LABEL_199',
  'mask': <PIL.Image.Image
           image mode=L size=3640x2226>},
 {'score': 0.999483,
  'label': 'cat',
  'mask': <PIL.Image.Image
           image mode=L size=3640x2226>}]
```

- **label · score** — 앞 실습과 동일
- **mask** — 해당 영역을 표시한 흑백 이미지
- 구분된 영역마다 하나씩 출력
- 좌표(box)가 아닌 영역 자체를 반환

## 세그멘테이션 결과 시각화

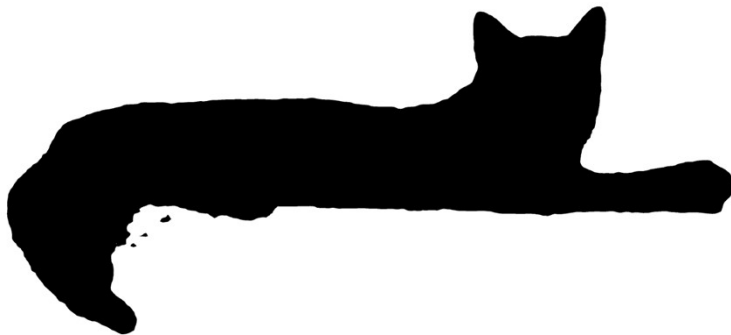
마스크를 원본 이미지에 겹쳐 영역 확인

### 개별 마스크 확인

```
print("구분된 영역:", len(result))

for r in result:
    print(r["label"], round(r["score"], 3))

result[0]["mask"]
```



### 원본에 겹치기

```
overlay = image.convert("RGBA")
mask = result[0]["mask"]

color = Image.new(
    "RGBA", image.size, (255, 0, 0, 100)
)
overlay.paste(color, (0, 0), mask)

overlay
```



## Foundation Model

# 제로샷 분류

학습에 사용되지 않은 분류 범주를 추론 시점에 지정하여 분류하는 방식

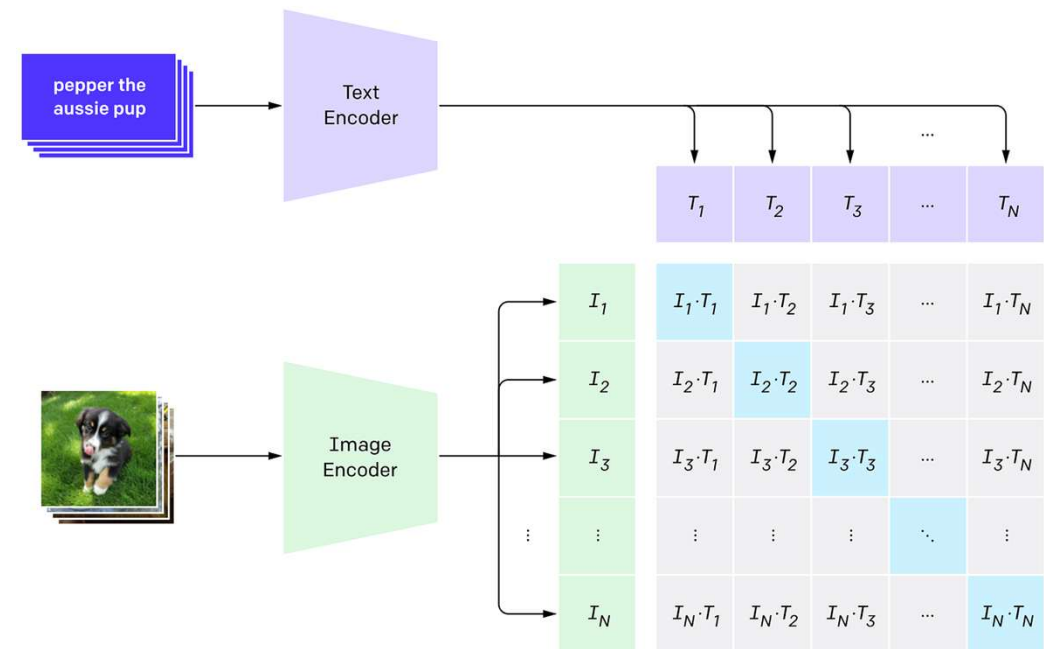
### 제로샷의 정의

- 제로샷(zero-shot) — 학습 예시를 하나도 사용하지 않음
- 분류 범주를 추론 시점에 텍스트로 지정
- 파라미터 변경 없음 — 추론만 수행
- 대비 — 소수샷(few-shot)은 소량의 예시 제공

### 작동 원리

- 이미지·텍스트를 함께 학습한 모델 (CLIP·SigLIP 등)
- 이미지·후보 문구를 각각 임베딩으로 변환
- 같은 공간에서 비교 — 유사도 산출
- 유사도가 가장 높은 후보를 결과로 반환

### 1. Contrastive pre-training





## 제로샷 분류 — 데이터 준비

### 데이터셋 불러오기

```
from datasets import load_dataset

ds = load_dataset(
    "imageomics/IDLE-00-Camera-Traps",
    split="test[:100]"
)
```

**datasets** — Hub 데이터셋 전용 라이브러리  
**첫 인자** — 데이터셋 이름 (제작자/데이터셋명)  
**split** — 사용할 구간과 개수 지정

### 데이터셋과 split

- 카메라 트랩 이미지 **2,586**장, **142**종 — LILA BC 기반
- 구간은 **test** 하나
- **"test[:100]"** — 앞 100장만 내려받기
- **"test[:5%]"** — 비율로도 지정 가능

## Foundation Model

# 제로샷 분류 — 데이터 준비

### 이미지와 후보 목록 확인

```
ds[0]["image"]  
  
print(ds[0]["common_name"])
```

**ds[0]** — 첫 번째 항목 선택

**image** — 화면에 이미지 표시

**common\_name** — 해당 이미지의 정답 종 이름



ds[0]["image"] 출력 — 카메라 트랩 사진

## 제로샷 분류 — 데이터 준비

### 데이터 구조 확인

#### 입력

ds

#### 출력

```
Dataset({
  features: ['image', 'original_label',
            'scientific_name', 'common_name',
            'kingdom', 'phylum', 'cls',
            'order', 'family', 'genus',
            'species'],
  num_rows: 100
})
```

#### 출력 항목

- **features** — 각 항목의 구성 열
- **num\_rows** — 불러온 이미지 수
- **주요 열** — image(이미지) · common\_name(일반명) · scientific\_name(학명)
- **그 외** — 분류 계층 (강·목·과·속·종)
- 실습에서는 **image**와 **common\_name** 사용

## 제로샷 분류 — 데이터 준비

### 이미지와 후보 목록 확인

#### 이미지와 정답

```
ds[0]["image"]  
  
print(ds[0]["common_name"])
```

- **ds[0]** — 첫 번째 항목 선택
- **image** — 화면에 이미지 표시
- **common\_name** — 해당 이미지의 정답 종 이름

#### 후보 목록 생성

```
labels = sorted(set(ds["common_name"]))  
  
print(len(labels))  
print(labels[:5])
```

- **set** — 중복 제거하여 종 목록 생성
- **sorted** — 이름순 정렬
- 이 목록을 다음 실습의 후보로 사용

## Foundation Model

# 실습 — 제로샷 분류

### 후보 목록을 지정하여 종을 예측

#### 코드

```
from transformers import pipeline

clf = pipeline(
    task="zero-shot-image-classification",
    model="google/siglip-base-patch16-224"
)

result = clf(ds[0]["image"], candidate_labels=labels)
result[:5]
```

- 앞 실습과 동일한 구조 — 작업 이름·모델만 변경
- **candidate\_labels** — 후보 목록 지정
- **ds · labels** — 앞 단계에서 준비한 데이터·목록

#### 출력 형태

```
[{'label': 'cheetah', 'score': 0.8721},
 {'label': 'spotted hyena', 'score': 0.0413},
 {'label': 'donkey', 'score': 0.0198},
 ...]
```

- **label · score** — 앞 실습과 동일한 형식
- 후보 목록의 모든 항목에 점수 부여
- 점수가 높은 순으로 정렬

# 예측과 정답 비교

### 정답 라벨과 대조하여 예측 성능 확인

#### 개별 확인

```
print("예측:", result[0]["label"])
print("정답:", ds[0]["common_name"])
print("점수:", result[0]["score"])
```

- `result[0]` — 점수가 가장 높은 예측
- 정답 열(`common_name`)과 나란히 출력

#### 여러 장 확인

```
n = 20
correct = 0

for item in ds.select(range(n)):
    pred = clf(item["image"],
                candidate_labels=labels)
    if pred[0]["label"] == item["common_name"]:
        correct += 1

print("정확도:", correct / n)
```

- `select` — 지정한 개수만 선택
- 예측과 정답이 일치한 횟수를 집계
- 학습을 전혀 하지 않은 상태의 성능

# 제로샷 분류

후보를 교체하면 동일 이미지에서 다른 판단 수행

### 코드

```
behaviors = [  
    "a picture of an animal eating",  
    "a picture of an animal moving",  
    "a picture of an animal resting",  
]  
  
clf(ds[0]["image"],  
    candidate_labels=behaviors)
```

### 출력

```
[{'score': 0.009415524080395699,  
  'label': 'a picture of an animal moving'},  
 {'score': 0.001487100962549448,  
  'label': 'a picture of an animal resting'},  
 {'score': 0.0006827009492553771,  
  'label': 'a picture of an animal eating'}]
```

### 확인 사항

- 동일 모델이 완전히 다른 질문에 응답
- 후보 목록이 곧 질문의 내용
- **정답 라벨 없음** — 사진을 보며 직접 판단
- 문구 형태에 따라 결과 변화 — 단어 vs 문장 비교

Foundation Model

# 추론 실습 2

---



## Foundation Model

# 실습 — 이미지 분류

### 7개 비전 과제 · 실습 데이터셋 구성

| 작업          | 데이터    | 내용                                        |
|-------------|--------|-------------------------------------------|
| ① 이미지 분류    | 철도 궤도  | 궤도 표면 촬영 — 결함 · 정상 2종 · 이진 분류             |
| ② 객체 탐지     | 혈액 도말  | 현미경 촬영 — 적혈구 · 백혈구 · 혈소판 3종, 세포별 위치 정답 포함 |
| ③ 세그멘테이션    | PCB 기판 | 인쇄회로기판 — 부품 · 배선 · 패드가 조밀하게 배치된 구조        |
| ④ 깊이 추정     | 일반 장면  | 실내외 일상 장면 — 하늘 · 도로 · 가구 등 150종           |
| ⑤ 이미지 설명 생성 | 일반 장면  | ④와 동일 데이터 — 앞선 전문 영상도 함께 입력               |
| ⑥ 문서 질의응답   | 재무 보고서 | 스캔 문서 — 예산서 · 명세서 등, 글자와 배치 정보 포함         |
| ⑦ 시각 질의응답   | 뇌 MRI  | 뇌종양 영상과 질문 · 답변 쌍                         |

# 이미지 분류

이미지 전체를 하나의 범주로 판별

### 작업

- 입력 — 이미지 한 장
- 출력 — 라벨(label)과 점수(score)
- 점수가 높은 순으로 정렬 — 상위 후보 함께 제시
- 판별 범위 — 모델이 학습한 분류 체계 내

### 데이터와 모델

- 궤도 표면을 촬영한 점검용 이미지
- 범주 2종 — Defective(결함) · Non-defective(정상)
- 정답 라벨 포함 — 예측과 대조 가능
- 모델 — ViT 분류 모델



Defective



Non-defective

## Foundation Model

# 실습 — 이미지 분류

### 궤도 이미지의 결함 여부 판별

```
from datasets import load_dataset
from transformers import pipeline

ds = load_dataset(
    "crangana/railroad-fault-detection",
    split="train[:100]"
)
labels = ds.features["label"].names
image = ds[0]["image"]

clf = pipeline(
    task="image-classification",
    model="crangana/railroad-fault-detector",
    device=0
)

clf(image)
```

#### 결과 확인

##### 출력 형태

```
[{'label': 'Defective', 'score': 0.97},
 {'label': 'Non-defective', 'score': 0.03}]
```

정답: Defective

##### 정확도 측정

```
correct = 0
for item in ds:
    pred = clf(item["image"])[0]["label"]
    if pred == labels[item["label"]]:
        correct += 1

print("정확도:", correct / len(ds))
```

# 객체 탐지

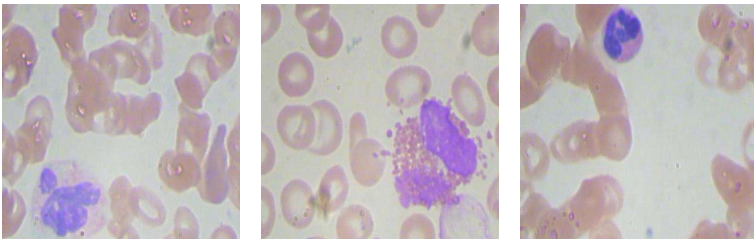
### 대상의 종류와 위치를 함께 판별

#### 작업

- 입력 — 이미지 한 장
- 출력 — 라벨 · 점수 · 위치 상자(box)
- 대상마다 하나씩 출력 — 개수 파악 가능
- 좌표 기준 — 이미지 좌측 상단이 (0, 0)

#### 데이터와 모델

- 현미경으로 촬영한 혈액 도말 이미지
- 범주 3종 — rbc(적혈구) · wbc(백혈구) · platelets(혈소판)
- 세포마다 위치 상자가 정답으로 포함
- 한 장에 세포 수십 개 — 개수 계산 가능
- 모델 — DETR 탐지 모델



```
{ "id": [ 2567, 2568, 2569, 2570, 2571, 2572, 2573, 2574, 2575, 2576 ], "area": [ 6321, 5187, 5278, 8512, 6321, 10384, 4440, 3932,
```

## Foundation Model

# 실습 — 객체 탐지

### 혈액세포의 종류와 위치 검출

#### 코드

```
ds = load_dataset(
    "keremberke/blood-cell-object-detection",
    name="full", split="train[:100]")
image = ds[0]["image"]

det = pipeline(
    task="object-detection",
    model="theodullin/detr-resnet-50_"
        "finetuned_blood_cell_15epochs",
    device=0
)

result = det(image, threshold=0.5)

print("검출:", len(result))
result[:3]
```

#### 결과 확인

##### 출력 형태

검출: 42

```
[{'label': 'rbc', 'score': 0.98,
  'box': {'xmin': 12, 'ymin': 45,
          'xmax': 78, 'ymax': 110}}, ...]
```

##### 범주별 개수

```
from collections import Counter
Counter(r["label"] for r in result)
{'rbc': 38, 'wbc': 2, 'platelets': 2}
```

라벨.점수는 분류와 동일 — box만 추가  
세포 종류별 개수를 자동 집계

### 예측 상자와 정답 상자의 대조

#### 코드

```
from PIL import ImageDraw

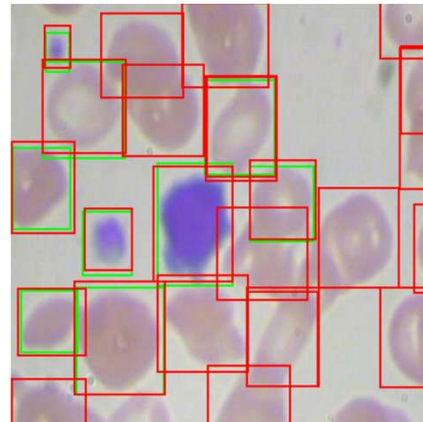
img = image.copy().convert("RGB")
draw = ImageDraw.Draw(img)

# 정답 - 초록
for box in ds[0]["objects"]["bbox"]:
    x, y, w, h = box
    draw.rectangle((x, y, x + w, y + h),
                   outline="lime", width=2)

# 예측 - 빨강
for r in result:
    b = r["box"]
    draw.rectangle((b["xmin"], b["ymin"],
                    b["xmax"], b["ymax"]),
                   outline="red", width=2)
```

#### 확인 사항

- **초록** — 데이터에 포함된 정답 위치
- **빨강** — 모델이 예측한 위치
- **두 상자가 겹칠수록** 정확한 예측
- **놓친 세포 · 잘못 잡은 영역 확인**



## Foundation Model

# 세그멘테이션

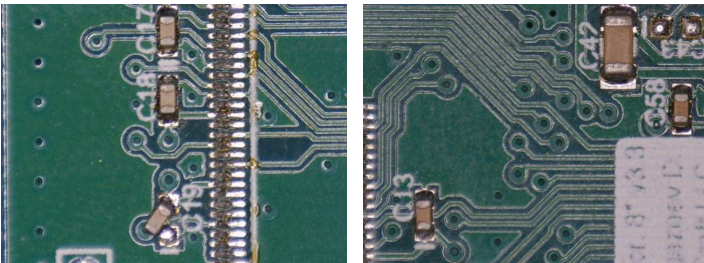
픽셀 단위로 영역의 경계를 구분

### 작업

- 입력 — 이미지 한 장
- 출력 — 영역 마스크(mask) — 대상 흰색, 배경 검은색
- 탐지와의 차이 — 사각형이 아닌 대상의 실제 윤곽
- 마스크는 이미지 형태 — 화면에 그대로 표시 가능

### 데이터와 모델

- 인쇄회로기판(PCB)을 촬영한 검사용 이미지
- 부품 · 배선 · 패드가 조밀하게 배치된 구조
- **모델 — SAM**(Segment Anything Model)
- SAM은 범주를 학습하지 않음 — 경계만 검출
- 결과에 이름이 없음 — 무엇인지는 판별하지 않음



# 실습 — 세그멘테이션

### PCB 기판의 영역 경계 검출

#### 코드

```
ds = load_dataset(
    "keremberke/pcb-defect-segmentation",
    name="full", split="train[:50]"
)
image = ds[0]["image"]

seg = pipeline(
    task="mask-generation",
    model="facebook/sam-vit-base",
    device=0
)

out = seg(image, points_per_batch=32)

print("검출된 영역:", len(out["masks"]))
out["masks"][0]
```

#### 결과 확인

##### 출력 형태

검출된 영역: 87

out["masks"]    - 마스크 목록 (흑백)  
out["scores"]   - 각 마스크의 신뢰도

##### 원본에 겹치기

```
from PIL import Image

overlay = image.convert("RGBA")
color = Image.new("RGBA", image.size,
                  (255, 0, 0, 100))
overlay.paste(color, (0, 0), out["masks"][0])
```

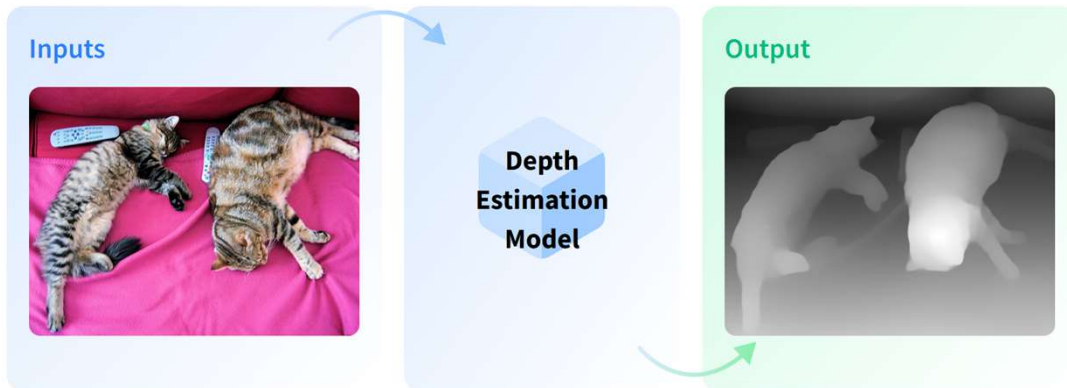
- 마스크 — 대상은 흰색, 나머지는 검은색
- 라벨이 없음 — 번호로만 구분



## Foundation Model

# 깊이 추정이란

사진 한 장으로 무엇이 가깝고 먼지를 판단하는 작업



### 정의

- 사진 한 장에서 원근을 추정 — 특수 장비 불필요
- **결과** — 밝을수록 가깝고 어두울수록 먼 흑백 이미지

### 주의할 점

- **상대 깊이** — 어느 쪽이 더 가까운지만 알 수 있음
- 실제 거리(m)는 알 수 없음 — 단위 없는 값
- 거리 측정이 목적이면 적합하지 않음

### 활용 분야

- **로봇 · 자율주행** — 장애물까지의 원근 파악
- **사진 편집** — 배경 흐림(인물 모드) 효과
- **3D 복원** — 사진에서 입체 구조 추정
- **증강현실** — 가상 물체를 장면에 배치

# 깊이 추정

단일 이미지에서 대상까지의 상대 거리를 추정

### 작업

- 입력 — 이미지 한 장
- 출력 — 깊이 지도(이미지) · 수치 배열
- 밝을수록 가까움 · 어두울수록 멀리
- 상대 깊이 — 단위 없음, 절대 거리 아님
- 작업 이름 — **depth-estimation**

### 데이터와 모델

- 실내외 일상 장면 이미지
- 150개 범주 — 하늘 · 도로 · 사람 · 가구 등
- 일반 장면 — 학습 데이터와 성격 유사
- 모델 — **DPT**(Dense Prediction Transformer)

## Foundation Model

# 실습 — 깊이 추정

### 일반 장면의 원근 추정

#### 코드

```
ds = load_dataset(
    "mervel/scene_parse_150",
    split="train[:50]"
)
image = ds[0]["image"]

depth = pipeline(
    task="depth-estimation",
    model="Intel/dpt-hybrid-midas",
    device=0
)

out = depth(image)
out["depth"]
```

#### 결과 확인

```
from PIL import Image

w, h = image.size
pair = Image.new("RGB", (w * 2, h))
pair.paste(image.convert("RGB"), (0, 0))
pair.paste(out["depth"].convert("RGB"), (w, 0))
pair
```



## Foundation Model

### 이미지 설명 생성이란

#### 이미지의 내용을 문장으로 서술하는 작업



A group of different animals are grazing in the wild.  
A group of wild animals in a field outside  
A herd of of wild animals walking around a field surrounded by forest.  
Zebras, giraffes and kudus on the run across a plain.  
it is a savannah with zebras and giraffes.

#### 정의

- 사진을 보고 내용을 문장으로 만들어 냄
- **분류와 차이** — 정해진 이름이 아닌 자유로운 문장

#### 주의할 점

- 생성 방식 — 실행할 때마다 표현이 달라질 수 있음
- 일상 사진으로 학습 — 전문 영상은 서술이 빈약
- 사실 확인 필요 — 그럴듯한 오답 가능

#### 활용 분야

- **접근성** — 시각장애인용 이미지 대체 텍스트
- **검색** — 사진에 설명을 붙여 검색 가능하게
- **자료 정리** — 대량 이미지에 설명 자동 부여

## Foundation Model

# 이미지 설명 생성

### 이미지의 내용을 문장으로 서술

#### 작업

- 입력 — 이미지 한 장
- 출력 — 생성된 문장 **generated\_text**
- 라벨이 아닌 자유 형식 서술 — 정해진 범주 없음
- 생성 방식 — 실행할 때마다 표현이 달라질 수 있음

#### 데이터와 모델

- 실내외 일상 장면 이미지 — ④와 동일 데이터
- 앞선 실습의 전문 영상(궤도 · 혈액 · 기판)도 함께 입력
- 모델 — 이미지 인코더 + 문장 생성기 결합 구조
- 일상 사진으로 학습 — 서술 범위가 그에 좌우

## Foundation Model

# 실습 — 이미지 설명 생성

### 일반 장면과 전문 영상의 서술 비교

#### 코드

```
ds = load_dataset(  
    "mervel/scene_parse_150",  
    split="train[:50]"  
)  
image = ds[0]["image"]  
  
cap = pipeline(  
    task="image-to-text",  
    model="nlpconnect/vit-gpt2-  
        image-captioning",  
    device=0  
)  
  
cap(image)
```

#### 결과 확인



a street vendor selling a variety of colorful items

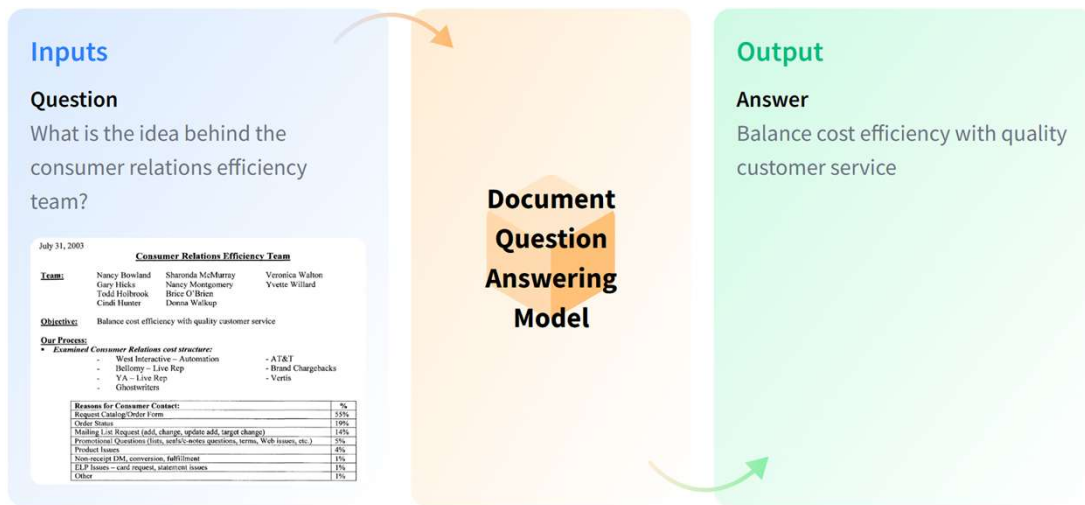


a broken down brick wall with a fire hydrant

# Foundation Model

## 문서 질의응답이란

문서 이미지에 질문하여 필요한 값만 뽑아내는 작업



### 정의

- 스캔 문서에 질문을 던져 답을 얻음
- 입력** — 문서 이미지 + 질문 문장
- 특징** — 글자와 배치를 함께 인식

### 주의할 점

- 문서에 없는 정보를 물으면 엉뚱한 답
- 표·서식이 복잡하면 정확도 하락

### 활용 분야

- 행정·회계** — 청구서·명세서에서 금액 추출
- 기록물 정리** — 스캔 자료에서 항목 추출
- 서류 처리** — 대량 문서에서 특정 값 수집

### 연구 활용

- 설문지·기록표 스캔본의 항목 수집
- 별도 OCR 도구 없이 바로 질문 가능

# 문서 질의응답

### 문서 이미지에 질문하여 답을 추출

#### 작업

- 입력 — 문서 이미지 + 질문
- 출력 — 답변 **answer**
- 문서의 글자와 배치를 함께 인식
- 문서에 없는 정보는 답할 수 없음

#### 데이터와 모델

- 재무 문서를 촬영 · 스캔한 이미지
- 예산서 · 명세서 · 보고서 등 업무 문서
- 문서 내 글자와 배치 정보 포함
- **모델 — Donut**(Document Understanding Transformer)
- OCR 도구 없이 이미지에서 직접 판독



## Foundation Model

# 실습 — 문서 질의응답

### 재무 문서에서 항목별 정보 추출

#### 코드

```
ds = load_dataset(
    "Anas989898/Vision-OCR-
    Financial-Reports-10k",
    split="train[:50]"
)
doc = ds[0]["image"]

qa = pipeline(
    task="document-question-answering",
    model="naver-clova-ix/donut-base-
    finetuned-docvqa",
    device=0
)

qa(image=doc,
    question="What is the total amount?")
```

#### 결과 확인

##### 출력 형태

```
[{'answer': '1,250,000'}]
```

##### 질문 바꿔보기

```
questions = [
    "What is the total amount?",
    "What is the date?",
    "Who is the issuer?",
]

for q in questions:
    print(q)
    print(" →", qa(image=doc,
        question=q)[0]["answer"])
```

- 같은 문서 · 다른 질문 → 다른 답변
- 문서에 없는 정보를 물으면 엉뚱한 답 산출

## Foundation Model

### 시각 질의응답이란

일반 이미지에 자유롭게 질문하여 답을 얻는 작업



### 정의

- 사진에 질문을 던져 답을 얻음
- **문서 질의응답과 차이** — 문서가 아닌 일반 사진
- **답변 형태** — 단어 수준의 짧은 답 + 점수

### 주의할 점

- 일상 사진으로 학습 — 전문 영상은 학습 범위 밖
- 모르는 것도 답을 내놓음 — 점수로 확신 정도 확인

### 활용 분야

- **접근성** — 사진 내용을 음성으로 안내
- **자료 검수** — 사진에 특정 요소가 있는지 확인
- **교육** — 이미지 기반 문답 자료 제작

# 시각 질의응답

임의의 이미지에 질문하여 답을 얻음

### 작업

- 입력 — 이미지 + 질문 (⑥과 동일 구조)
- 출력 — 답변(answer)과 점수(score)
- 문서가 아닌 일반 이미지 대상
- 짧은 질문에 단어 수준으로 응답

### 데이터와 모델

- 뇌종양 MRI 이미지와 질문 · 답변 쌍
- 데이터에 정답 답변 포함 — 모델 답변과 대조 가능
- **모델** — ViLT(Vision-and-Language Transformer)
- 일상 이미지로 학습 — 의료 영상은 학습 범위 밖

## Foundation Model

# 실습 — 시각 질의응답

### 의료 영상에 질문하여 답변 확인

#### 코드

```
ds = load_dataset(
    "Kaith-jeet123/brain_tumor_vqa",
    split="train[:50]"
)
image = ds[0]["image"]

vqa = pipeline(
    task="visual-question-answering",
    model="dandelin/vilt-b32-
        finetuned-vqa",
    device=0
)

vqa(image=image,
    question="Is there a tumor?")
```

#### 결과 확인

##### 출력 형태

```
[{'answer': 'yes', 'score': 0.71},
 {'answer': 'no', 'score': 0.18},
 {'answer': 'maybe', 'score': 0.04}]
```

##### 일반 질문과 비교

```
for q in ["What color is this?",
    "How many objects are there?",
    "Is this a medical image?"]:
    print(q, "→",
        vqa(image=image,
            question=q)[0]["answer"])
```

Foundation Model

# 이미지 미세 조정

---

# 미세조정

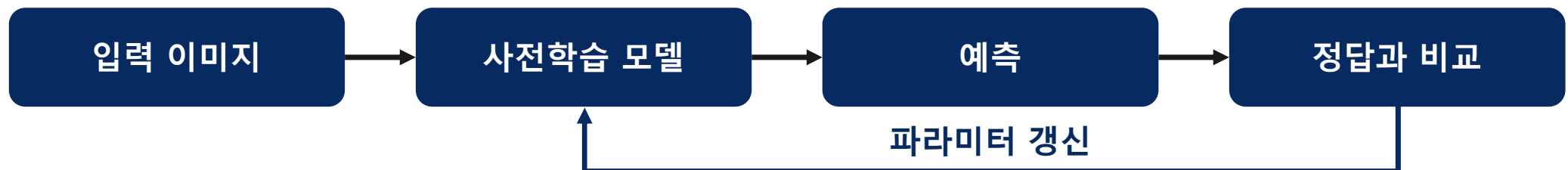
사전학습 모델의 파라미터를 보유 데이터로 갱신하여 특정 작업에 적응시키는 과정

### 미세조정의 정의

- 사전학습 모델을 출발점으로 사용 — 무작위 초기화에서 시작하지 않음
- 보유 데이터로 **파라미터 갱신** 수행
- **지도학습 방식** — 정답 라벨을 기준으로 예측과의 차이를 조정
- 결과물 — 해당 작업에 **특화된 모델**

### 처음부터 학습과의 차이

- **처음부터 학습** — 무작위 상태에서 시작, 대량 데이터·장시간 소요
- **미세조정** — 사전학습으로 확보한 범용 표현을 재사용
- **소량 데이터**로도 특정 작업 적응 가능
- **학습 시간·계산 자원 절감**



## 추론과 미세조정

### 파라미터 갱신 여부에 따른 구분

#### 추론

- **파라미터 고정** — 모델이 변하지 않음
- **입력** — 이미지만
- **수행** — 예측 산출 (순전파 1회)
- **결과물** — 예측값, 모델은 그대로
- **도구** — **pipeline**

#### 미세조정

- **파라미터 갱신** — 모델이 변함
- **입력** — 이미지 + 정답 라벨
- **수행** — 예측·비교·조정의 반복
- **결과물** — 특화된 새 모델
- **도구** — **Trainer**

# 학습 범위에 따른 구분

### 갱신 대상 파라미터의 범위에 따른 두 방식

|         | 전체 미세조정 | PEFT       |
|---------|---------|------------|
| 갱신 대상   | 모든 파라미터 | 일부·추가 파라미터 |
| 원본 파라미터 | 변경      | 대부분 유지     |
| 학습 비용   | 높음      | 낮음         |
| 필요 데이터  | 많음      | 적음         |
| 저장 용량   | 원본과 동일  | 변경분만 별도 보관 |

### 전체 미세조정

- 모델 전체를 보유 데이터에 맞추어 조정
- 데이터가 충분한 경우 성능 우수

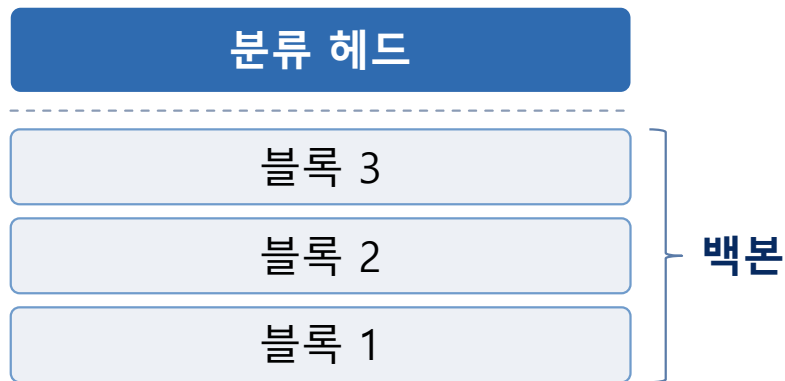
### PEFT (Parameter-Efficient Fine-Tuning)

- 소수의 파라미터만 학습하는 방식의 총칭
- 부분 미세조정 · LoRA 등이 이에 해당



## 전체 미세조정

사전학습 파라미터 전체를 보유 데이터로 갱신



- **백본(backbone)** — 이미지의 특징을 추출하는 부분, 블록의 누적
- **분류 헤드(head)** — 추출된 특징으로 최종 판단을 내리는 부분
- **전체 미세조정** — 백본과 헤드 모두 갱신

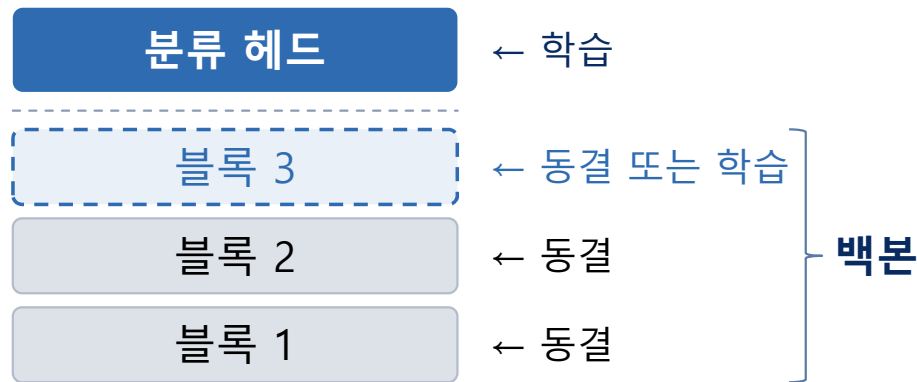
### 방식과 특징

- **사전학습 가중치를 초기값으로 사용**
- **분류 헤드 교체** — 보유 분류 체계에 맞게 새로 구성
- **성능** — 데이터가 충분한 경우 가장 우수
- **비용** — 학습 시간·메모리 소요 큼
- **데이터** — 소량인 경우 과적합 위험
- **결과물** — 원본과 동일한 크기의 새 모델

## Foundation Model

### 부분 미세조정

백본의 일부 또는 전부를 동결하고 나머지만 학습



#### 방식과 특징

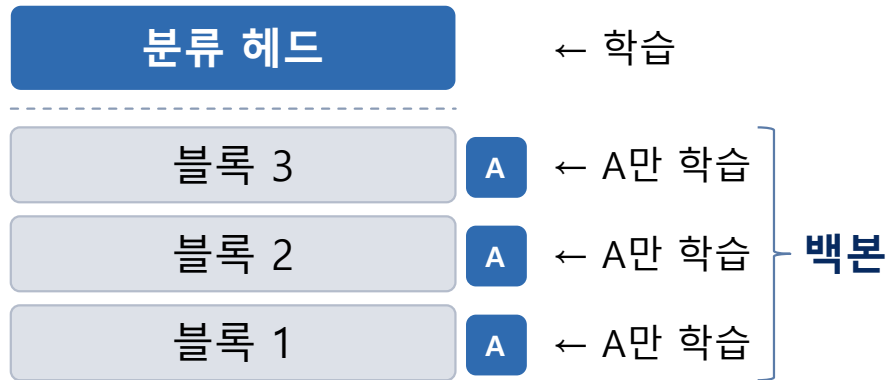
- **근거** — 백본은 형태·질감 등 범용 표현을 이미 보유
- **상위 블록** — 작업에 특화된 표현, 해제 시 성능 향상 여지
- **비용** — 학습 대상에 비례, 헤드만 학습 시 매우 빠름
- **데이터** — 소량으로도 수행 가능
- **결과물** — 동결된 부분은 공유, 학습한 부분만 별도 보관

- **동결(freeze)** — 파라미터를 갱신하지 않음
- **최소 범위** — 헤드만 학습, 선형 프로빙(linear probing)
- **확장** — 상위 블록부터 순차적으로 해제
- **하위 블록일수록 범용 표현 보유** — 동결 유지

## Foundation Model

# LoRA

원본 파라미터를 유지한 채 소형 행렬을 추가하여 학습



### 방식과 특징

- **학습 파라미터** — 전체의 수 % 이내
- **성능** — 동일 조건에서 전체 미세조정에 근접
- **저장** — 원본은 공유, 추가분만 별도 보관 (수십 MB 수준)
- **교체** — 용도별 모듈을 갈아 끼워 사용 가능
- **원본 보존** — 사전학습 표현이 변경되지 않음

- **A** — 추가 학습 모듈
- **원본 블록** — 전부 동결
- 각 블록 내부의 특정 가중치에 소형 행렬 삽입
- 학습 대상 — 추가한 행렬뿐
- 추론 시 원본 출력과 추가분을 합산

## Hugging Face의 미세조정

**Trainer** — 사전학습 모델을 학습에 사용하는 표준 도구

### Trainer의 정의

- 학습 루프(training loop)를 표준화한 구현체
- 미니배치 · 순전파 · 역전파 · 갱신을 자동 수행
- 지정 항목 — 모델 · 하이퍼파라미터 · 데이터셋
- 하이퍼파라미터는 **TrainingArguments**로 정의

### 학습 과정

- ① 미니배치 구성 — 학습 데이터를 일정 크기로 분할
- ② 순전파 — 입력에서 출력 방향으로 계산, 예측 산출
- ③ 손실 계산 — 손실 함수로 예측과 정답의 차이 측정
- ④ 역전파 — 손실에 대한 파라미터의 기울기 계산
- ⑤ 파라미터 갱신 — 옵티마이저가 학습률에 따라 조정

#### pipeline

추론  
순전파 1회  
작업 · 모델 · 입력  
고정

#### Trainer

학습  
순전파 + 역전파 반복  
모델 · 하이퍼파라미터 · 데이터셋  
갱신

용도  
수행 연산  
지정 항목  
모델 파라미터

Foundation Model

# 이미지 미세 조정 실습

---

# 실습 절차 비교

### 추론 실습과 비교한 미세조정 실습의 추가 절차

#### 추론 실습

- ① 이미지 준비
- ② 모델 선택
- ③ pipeline 실행
- ④ 결과 확인

- **필요 항목** — 이미지와 모델
- **정답 라벨** 불필요
- **소요 시간** — 즉시

#### 미세조정 실습

- ① 데이터 준비 ← 정답 라벨 필요
- ② 구간 분리 ← 학습용 / 평가용
- ③ 전처리 ← 모델 입력 형식 통일
- ④ 모델 구성 ← 백본 + 새 분류 헤드
- ⑤ 학습 범위 지정 ← 백본 동결
- ⑥ 학습 조건 설정 ← 반복 횟수 · 학습률
- ⑦ 학습 실행 ← Trainer

- **필요 항목** — 이미지 · 정답 라벨 · 모델
- **평가용 데이터** 별도 확보
- **소요 시간** — 학습 대기 발생

## Foundation Model

# 미세조정 실습 구성

같은 데이터 · 같은 절차 — 학습 범위만 달리하여 두 방식을 비교

| 실습        | 데이터               | 학습 대상          | 학습 비율 | 확인 사항        |
|-----------|-------------------|----------------|-------|--------------|
| ① 부분 미세조정 | Food-101 · 5,000장 | 백본 동결 + 새 헤드   | 0.09% | 정확도 1% → 79% |
| ② LoRA    | ①과 동일             | 백본에 LoRA 모듈 삽입 | 0.43% | ①과 성능 비교     |

### 왜 미세조정이 필요한가

- 사전학습 모델의 분류 체계에 없는 대상은 추론만으로 판별 불가
- **실습에서 확인** — 요리 사진에 접시 · 국자 등 주변 사물로 응답
- 보유 데이터로 소량만 학습시켜 해당 분류를 수행하게 만들



## Food-101 데이터셋 불러오기와 구간 분리

### 실습 데이터셋 준비

#### 코드

```
from datasets import load_dataset

ds = load_dataset(
    "ethz/food101",
    split="train[:5000]"
)

ds = ds.train_test_split(
    test_size=0.2, seed=42
)
ds
```

- 요리 사진 분류 데이터셋 — 101개 범주
- 전체 10만 장 중 **5,000장**만 사용
- **train\_test\_split** — 학습용·평가용 분리
- **seed=42** — 동일한 분할 재현  
(전원 동일 결과)

#### 출력

```
DatasetDict({ train: 4000장 · test: 1000장 })
```



# 범주 이름과 정답 구조 확인

### 라벨 확인

#### 코드

```
labels = ds["train"].features["label"].names

print("범주 수:", len(labels))
print("정답:",
      labels[ds["train"][0]["label"]])

ds["train"][0]["image"]
```

- `features["label"].names` — 범주 이름 목록
- 정답은 번호로 저장 — 이름 목록과 대조 필요
- 이미지와 정답을 눈으로 확인
- 이후 예측 결과 해석의 기준

# 사전학습 분류 모델의 예측 결과

미세조정 전 — 예측 확인

### 코드

```
from transformers import pipeline

clf = pipeline(
    task="image-classification",
    model="google/vit-base-patch16-224"
)

clf(ds["test"][0]["image"])
```

ImageNet 1000종으로 학습된 분류 모델

### 결과

```
[{'label': 'plate',      'score': 0.31},
 {'label': 'soup bowl', 'score': 0.18},
 {'label': 'ladle',     'score': 0.07}]
```

정답: beignets

- 요리 이름이 아닌 라벨 출력
- **원인** — 학습 분류 체계에 요리 범주 부재
- 접시·국자 등 주변 사물로 예측

# 모델 입력 형식에 맞춘 이미지 변환

## 전처리

### 코드

```
from transformers import AutoImageProcessor

ckpt = "google/vit-base-patch16-224-in21k"
processor = AutoImageProcessor.from_pretrained(ckpt)

def transform(batch):
    images = [img.convert("RGB") for img in batch["image"]]
    out = processor(images, return_tensors="pt")
    out["labels"] = batch["label"]
    return out

ds = ds.with_transform(transform)
```

- **ViT-in21k** — ImageNet-21k로 사전학습된 백본 (분류 헤드 없음)
- **AutoImageProcessor** — 모델별 전처리 규칙 자동 적용
- **수행** — 크기 조정 · 정규화 · 형식 변환
- **convert("RGB")** — 흑백 이미지도 3채널로 통일
- **with\_transform** — 사용 시점에 변환, 메모리 절약

## Foundation Model

# 모델 구성

### 사전학습 백본과 새 분류 헤드로 모델 구성

#### 코드

```
from transformers import \
    AutoModelForImageClassification

model = AutoModelForImageClassification \
    .from_pretrained(
        ckpt,
        num_labels=len(labels),
        id2label={i: n for i, n in enumerate(labels)},
        label2id={n: i for i, n in enumerate(labels)},
    )
```

#### 헤드 확인

```
print(model.classifier)
Linear(in_features=768, out_features=101)
```

- **AutoModelForImageClassification** — 백본 + 분류 헤드 구조를 자동 구성
- **백본** — 체크포인트의 사전학습 가중치로 채워짐
- **분류 헤드** — 채울 값이 없어 무작위 초기화
- **num\_labels** — 헤드의 출력 개수 (101)
- **id2label · label2id** — 번호와 이름의 대응
- **확인** — 768(백본 출력) → 101(요리 범주)

## Foundation Model

# 학습 전 모델의 평가 데이터 정확도

### 미세조정 전 — 정확도 측정

#### 코드

```
import torch

def accuracy(model):
    correct = 0
    for item in ds["test"]:
        x = item["pixel_values"].unsqueeze(0)
        with torch.no_grad():
            pred = model(x).logits.argmax(-1).item()
        if pred == item["labels"]:
            correct += 1
    return correct / len(ds["test"])

print("학습 전 정확도:", accuracy(model))
```

#### 결과와 의미

학습 전 정확도: 0.012

- 헤드가 무작위 초기화 상태 — 예측이 무작위 수준
- 101종 중 찍기 확률  $\approx 1\%$
- 결과가 이와 유사 — 아직 학습되지 않았음을 확인
- 이 값이 미세조정 성과의 기준선

# 백본 동결과 학습 대상 확인

### 학습 범위 지정

#### 코드

```
for p in model.vit.parameters():
    p.requires_grad = False

total = sum(p.numel() for p in model.parameters())
train = sum(p.numel() for p in model.parameters()
            if p.requires_grad)

print(f"전체: {total:,}")
print(f"학습: {train:,} ({train/total:.2%})")
```

#### 결과와 의미

전체: 85,875,173  
학습: 77,669 (0.09%)

- **requires\_grad = False** — 파라미터를 갱신하지 않음
- **model.vit** — 백본 부분
- **학습 대상** — 분류 헤드뿐
- 앞서 다룬 부분 미세조정 방식

### 학습 설정

#### 코드

```
from transformers import TrainingArguments

args = TrainingArguments(
    output_dir="food_model",
    num_train_epochs=3,
    per_device_train_batch_size=32,
    learning_rate=1e-3,
    remove_unused_columns=False,
    logging_steps=20,
)
```

- **num\_train\_epochs** — 전체 데이터 반복 횟수
- **per\_device\_train\_batch\_size** — 한 번에 처리할 이미지 수
- **learning\_rate** — 한 번에 조정하는 정도
- **remove\_unused\_columns=False** — 이미지 열 유지에 필수
- **output\_dir** — 결과 저장 폴더

# Trainer를 통한 학습 수행

### 학습 실행

#### 코드

```
from transformers import Trainer

trainer = Trainer(
    model=model,
    args=args,
    train_dataset=ds["train"],
    eval_dataset=ds["test"],
)

trainer.train()
```

#### 진행 표시 읽기

| Step | Training Loss |
|------|---------------|
| 20   | 4.31          |
| 40   | 3.52          |
| 60   | 2.98          |

- **Loss(손실)** — 예측과 정답의 차이
- 값이 감소하면 학습이 진행 중
- 감소가 멈추면 추가 학습 효과 미미
- 진행률 막대로 남은 시간 확인



## Foundation Model

# 학습 전후 예측과 정확도 비교

### 미세조정 후 — 결과 비교

#### 예측

```
clf_new = pipeline(  
    "image-classification",  
    model=model,  
    image_processor=processor)  
  
clf_new(ds["test"][0]["image"])
```

```
beignets 0.87 / churros 0.05  
정답: beignets
```

#### 정확도

```
print("학습 후 정확도:",  
      accuracy(model))
```

```
학습 전: 0.012  
학습 후: 0.79
```

- 같은 이미지 — 엉뚱한 라벨에서 정답으로
- 1,000장 정확도 — 1% → 79%
- 분류 헤드만 학습한 결과

# 학습한 모델의 저장 및 활용

### 모델 저장과 재사용

#### 저장

```
trainer.save_model("food_model")  
processor.save_pretrained("food_model")
```

- 모델과 전처리 규칙을 함께 저장
- 폴더 하나로 관리

#### 재사용

```
clf = pipeline(  
    task="image-classification",  
    model="food_model"  
)  
  
clf("my_photo.jpg")
```

- 저장한 폴더를 모델 이름 자리에 지정
- 이후에는 추론과 동일한 방식으로 사용
- Hub 업로드 — `push_to_hub("계정명/모델명")`

# LoRA 실습

앞 실습과 동일한 데이터·절차, 모델 구성만 변경

### 코드

```
from peft import LoraConfig, get_peft_model

model = AutoModelForImageClassification \
    .from_pretrained(
        ckpt,
        num_labels=len(labels),
        id2label=id2label, label2id=label2id,
    )
```

- **peft** — PEFT 기법 전용 라이브러리
- LoRA · 어댑터 · 프롬프트 튜닝 등 지원
- 학습되지 않은 모델을 다시 구성 — 동일 조건 비교

### 앞 실습과의 차이

그대로 사용

- **데이터셋** — 동일한 구간 분리 결과
- **전처리** — 동일한 processor
- **학습 도구** — Trainer, 동일한 절차

달라지는 부분

- **모델 구성** — LoRA 모듈 삽입
- **학습 범위** — 헤드 + 백본 내부 모듈
- 동결 코드 불필요 — 자동 처리

### 삽입 위치와 규모 지정

#### 코드

```
config = LoraConfig(
    r=16,
    lora_alpha=16,
    target_modules=["query", "value"],
    lora_dropout=0.1,
    bias="none",
    modules_to_save=["classifier"],
)

model = get_peft_model(model, config)
```

- **LoraConfig** — LoRA 적용 조건 정의
- **get\_peft\_model** — 원본 모델에 모듈을 삽입한 형태로 변환

#### 주요 항목

- **r** — 추가 모듈의 규모, 클수록 표현력·파라미터 증가
- **target\_modules** — 삽입 위치, 어텐션의 query · value
- **lora\_alpha** — 추가분의 반영 강도
- **lora\_dropout** — 과적합 방지 비율
- **modules\_to\_save** — LoRA 외에 함께 학습할 부분
  - 분류 헤드는 새로 생성되었으므로 반드시 포함

# LoRA 실습

### 학습 파라미터 비율 확인

#### 코드

```
model.print_trainable_parameters()
```

#### 결과

```
trainable params: 372,485  
all params: 86,247,658  
trainable%: 0.4319
```

- **print\_trainable\_parameters** — 학습 대상 파라미터 수와 비율 출력
- **학습 대상** — 삽입한 LoRA 모듈 + 분류 헤드
- 원본 백본은 동결 상태

#### 방식별 비교

| 방식          | 학습 대상          | 비율           |
|-------------|----------------|--------------|
| 전체 미세조정     | 전체             | 100%         |
| 부분 미세조정     | 헤드만            | 0.09%        |
| <b>LoRA</b> | <b>모듈 + 헤드</b> | <b>0.43%</b> |

- 부분 미세조정보다 학습 대상 증가
- 백본 내부까지 조정 — 성능 향상 여지
- **전체 미세조정 대비 200분의 1 수준**
- **r** 값 조정으로 학습 규모 변경 가능

앞 실습과 동일한 절차로 학습 수행

### 코드

```
args = TrainingArguments(  
    output_dir="food_lora",  
    num_train_epochs=3,  
    per_device_train_batch_size=32,  
    learning_rate=5e-4,  
    remove_unused_columns=False, logging_steps=20,  
)  
trainer = Trainer(  
    model=model, args=args,  
    train_dataset=ds["train"], eval_dataset=ds["test"]  
)
```

### 설정 비교

- 학습 절차 — 앞 실습과 동일
- 데이터·전처리 — 변경 없음
- **learning\_rate** — 헤드만 학습할 때보다 낮게 설정
- **근거** — 백본 내부까지 조정하므로 급격한 변화 방지
- **그 외 항목** — 동일하게 유지하여 조건 통제

### 방식별 성능과 비용 비교

#### 정확도 측정

```
print("LoRA 정확도:", accuracy(model))
```

#### 저장

```
model.save_pretrained("food_lora")
```

- 앞 실습과 동일한 함수로 측정 — 동일 기준 비교
- 저장 시 추가분만 보관 (수 MB)

#### 종합 비교

| 방식      | 학습 대상 | 정확도   | 저장   |
|---------|-------|-------|------|
| 학습 전    | —     | 0.01  | —    |
| 부분 미세조정 | 0.09% | 0.92  | 수 MB |
| LoRA    | 0.43% | 0.947 | 수 MB |

- 학습 범위 확대에 따른 성능 변화 확인
- 소요 시간과 성능의 균형 판단
- 데이터 규모가 작으면 차이가 작을 수 있음

## SAM 미세조정

마스크 디코더만 학습하여 의료 영상에 적응

### SAM의 특성

- 프롬프트 기반 — 점 · 박스로 대상을 지정하여 분할
- 사전학습 규모 — 이미지 1,100만 장, 마스크 10억 개
- 범용성 — 범주를 학습하지 않아 어떤 영상에도 적용
- 한계 — 의료 영상은 학습에 포함되지 않음

#### 분류가 아닌 분할 작업

- 출력이 라벨이 아니라 마스크
- 헤드 교체 대신 디코더 학습

### 학습 범위

비전 인코더

89.7M 동결

프롬프트 인코더

6.2K 동결

마스크 디코더

4.1M 학습

#### 학습 대상 — 전체의 약 4.3%

- 인코더는 이미 풍부한 시각 특징 보유
- 도메인 적응은 디코더에서 발생

앞서 다룬 부분 미세조정의 실제 적용 사례  
도입부 연구 사례(H-SAM · MedSAM)와 동일한 방식



## 실습 — SAM 미세조정

### 정답 마스크에서 박스 프롬프트 추출

#### 코드

```
from datasets import load_dataset
from transformers import SamProcessor

ds = load_dataset(
    "kowndinya23/Kvasir-SEG", split="train[:400]")

processor = SamProcessor.from_pretrained(
    "facebook/sam-vit-base")

item = ds[0]
item["image"] # 내시경 이미지
item["annotation"] # 정답 마스크
```

#### 박스 프롬프트 추출

```
box = get_bounding_box(mask, shift=20)
# [x_min, y_min, x_max, y_max]
```

#### Kvasir-SEG

- 대장내시경 용종 이미지 1,000장
- 의료 전문가가 픽셀 단위로 주석
- 용종 크기 편차 큼 — 화면의 1%에서 50%까지
- 조기 대장암 진단과 직결

#### 박스 프롬프트

- SAM은 프롬프트를 받아 분할 — 점 · 박스 · 마스크
- 학습 시 정답 마스크에서 박스를 자동 추출
- $\pm 20\text{px}$  무작위 흔들기 — 부정확한 입력에 대비
- 평가 시에는 흔들지 않음 (shift=0) — 공정한 비교

## 실습 — SAM 미세조정

### 마스크 디코더만 학습 후 전후 비교

#### 코드

```
model = SamModel.from_pretrained(
    "facebook/sam-vit-base").to(device)

# 인코더 동결
for name, p in model.named_parameters():
    if name.startswith(("vision_encoder",
                        "prompt_encoder")):
        p.requires_grad_(False)

loss_fn = DiceBCELoss(dice_weight=0.5)
optimizer = AdamW(
    model.mask_decoder.parameters(), lr=5e-5
)
```

- **Trainer 미사용** — 학습 루프 직접 작성
- 손실 = Dice + BCE 결합

#### 두 손실을 함께 쓰는 이유

- 용종은 화면의 일부 — 배경 픽셀이 압도적
- **BCE만 사용 시**
  - “대부분 배경”으로 기울어 경계가 뭉개짐
- **Dice** — 겹치는 영역을 직접 최적화

#### 결과 비교 (20장 평균)

미세조정 전 Dice: **0.6296**

미세조정 후 Dice: **0.8267**